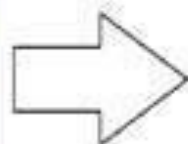

Computer Vision and Image Processing





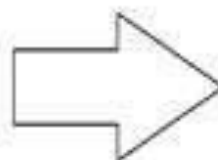
3d world around us



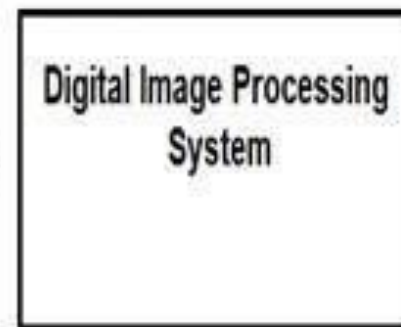
captured
by



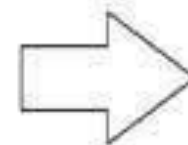
a camera



and sent to



a particular system to
focus on a water drop,



that's gives its
ouput as an



Processed image

Visual Perception

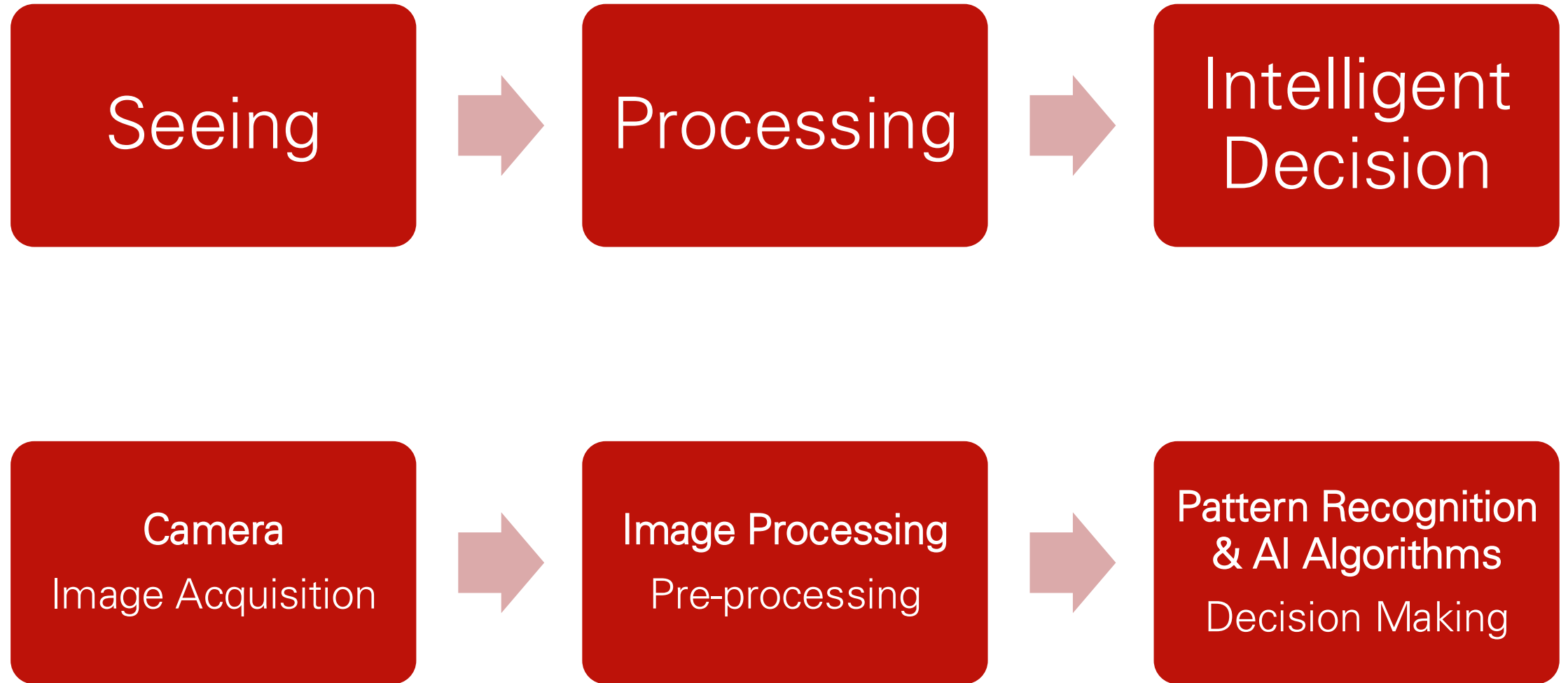
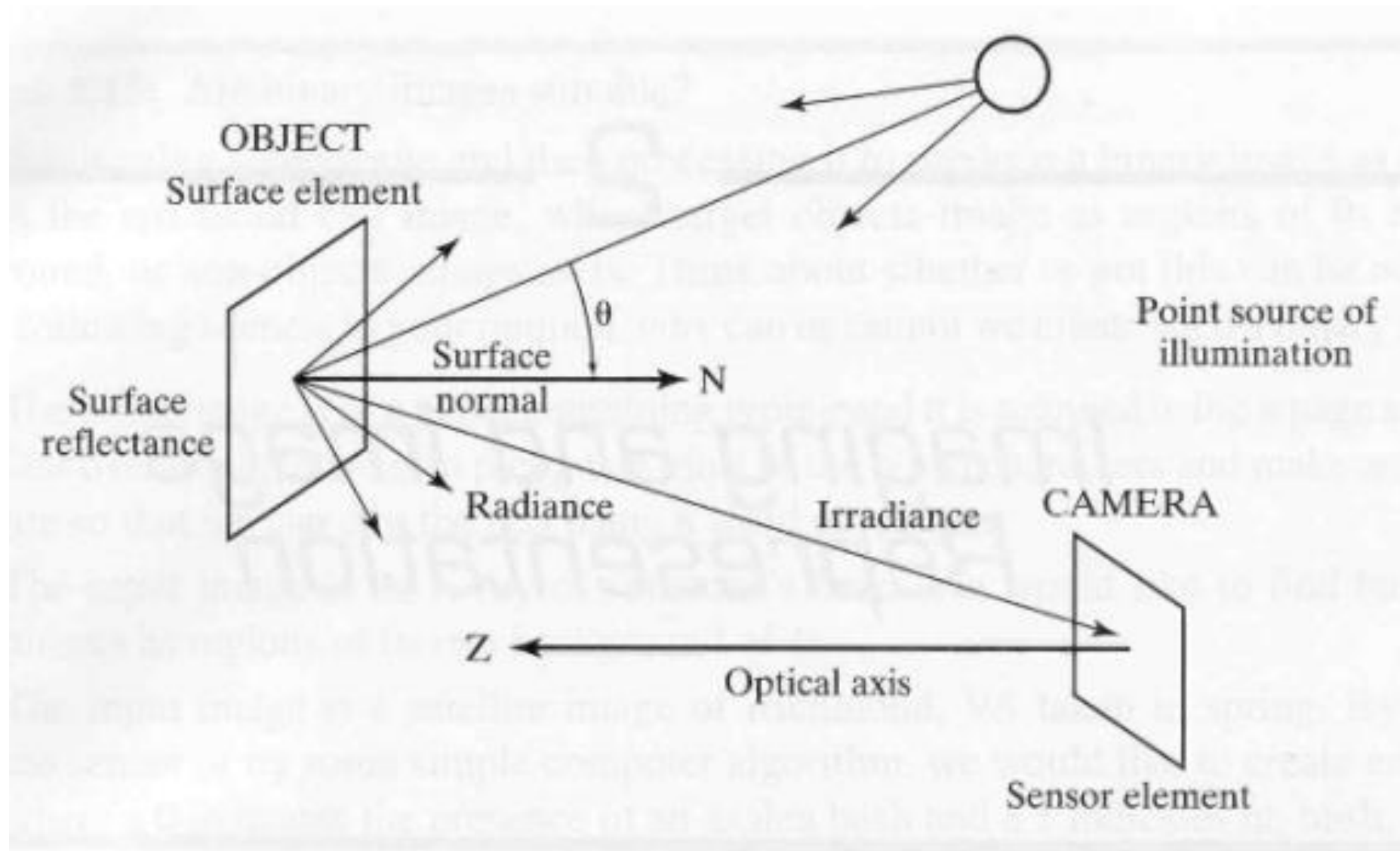
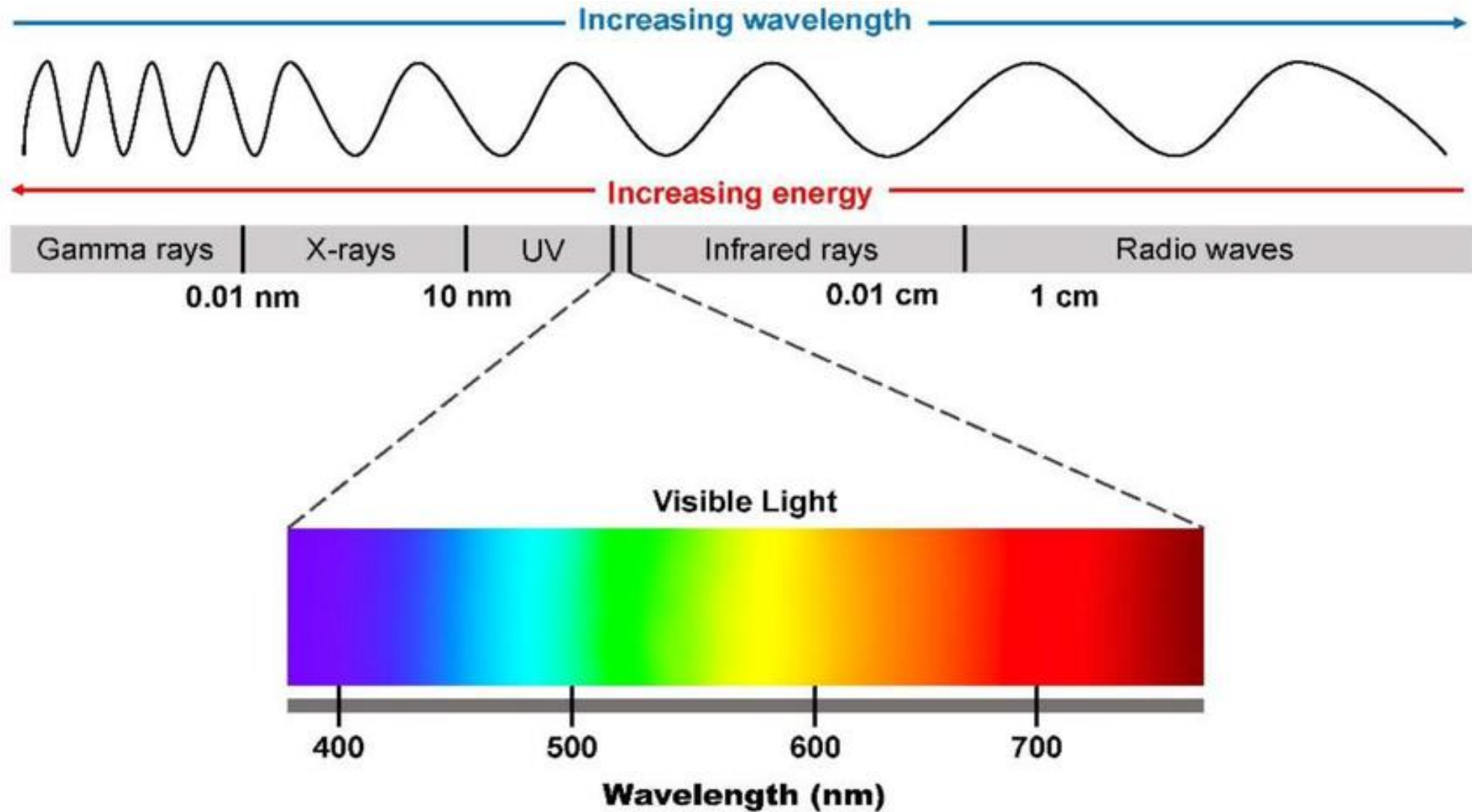


Image Formation



Source of Illumination





X-rays



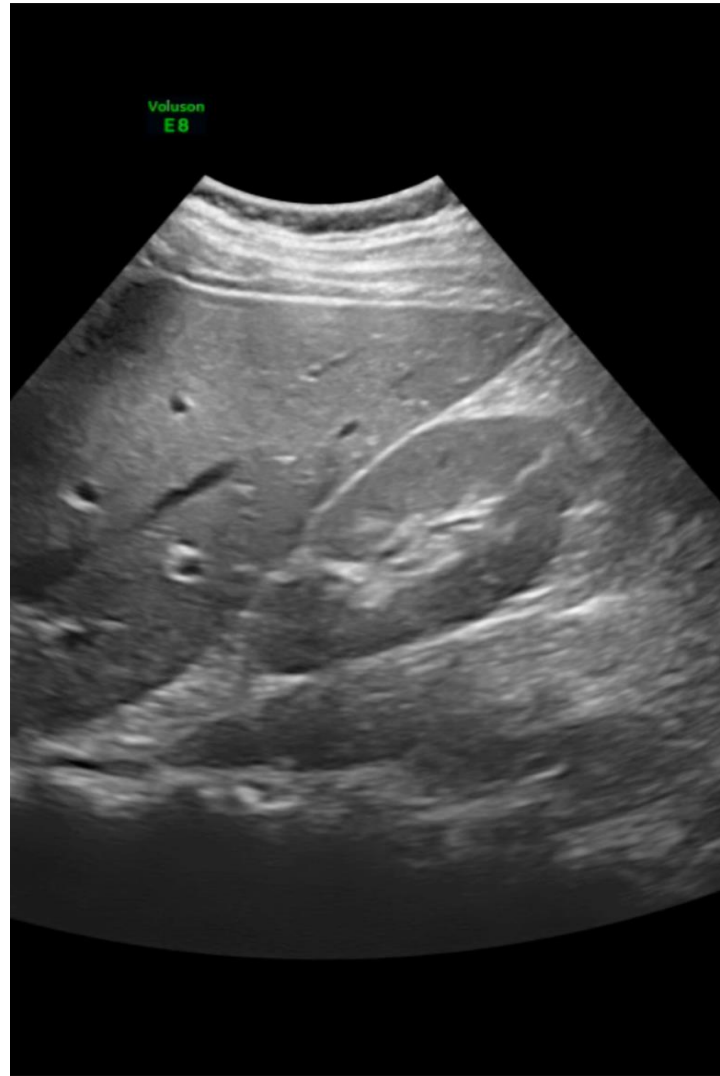
Visible Region



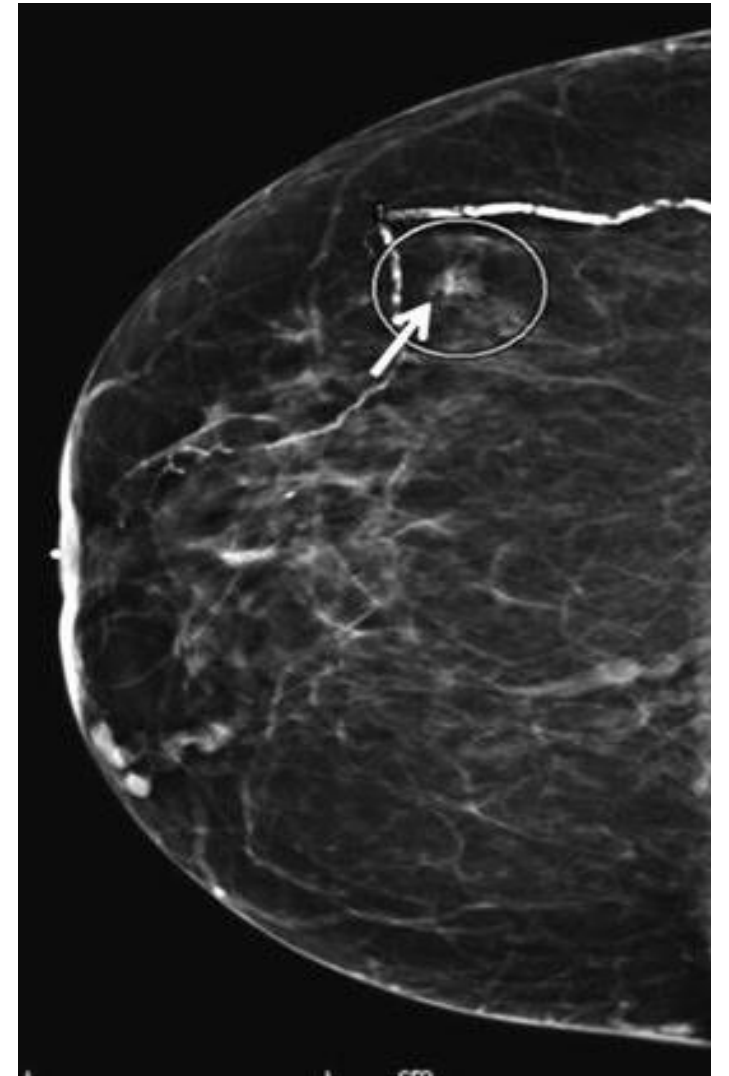
Infrared



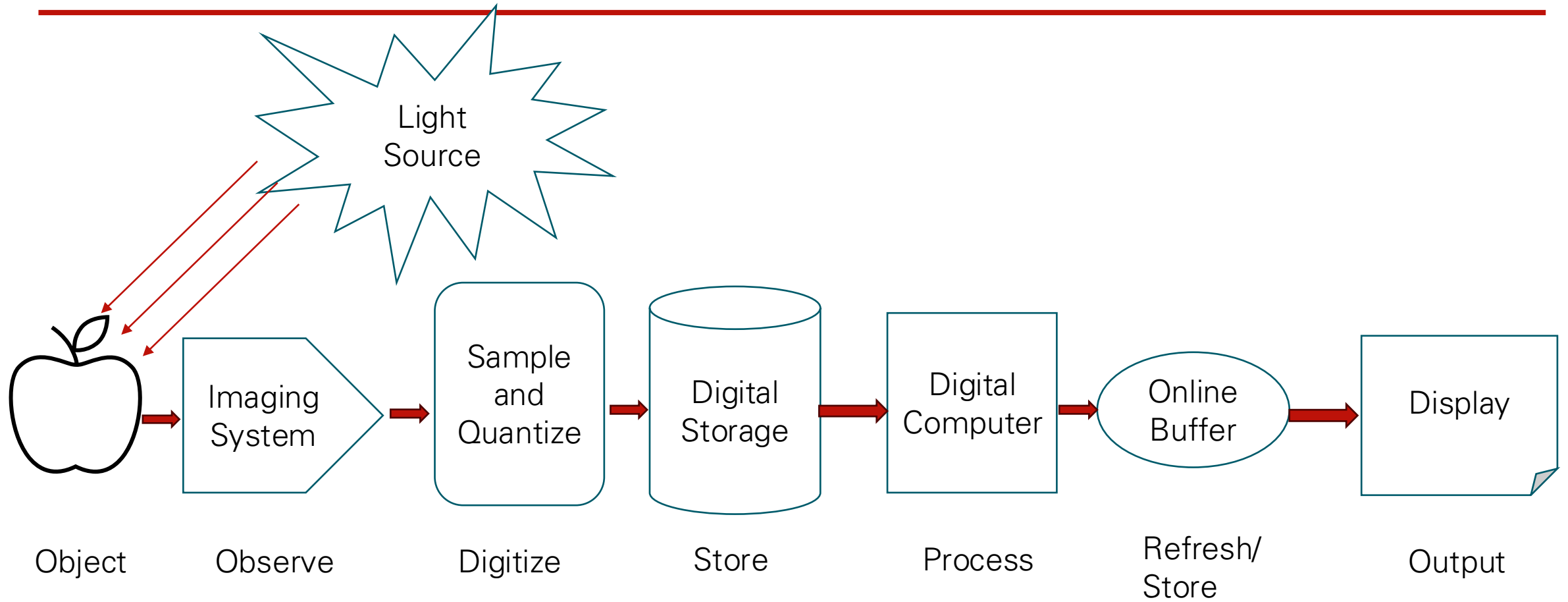
Synthetic Aperture
Radar



Ultrasound

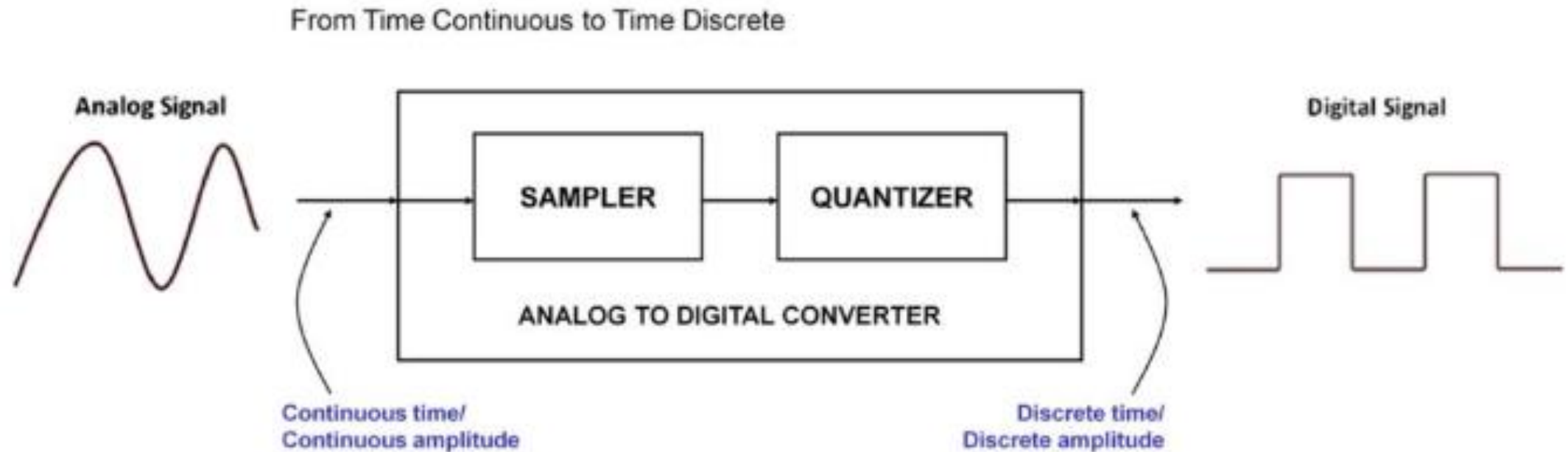


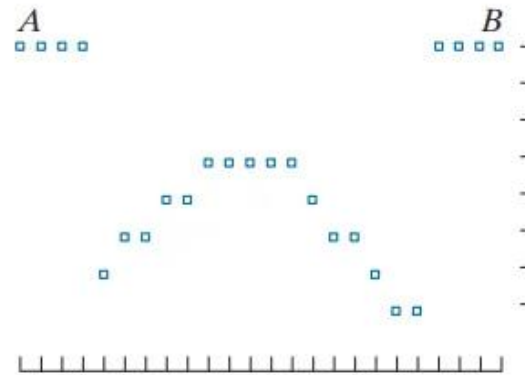
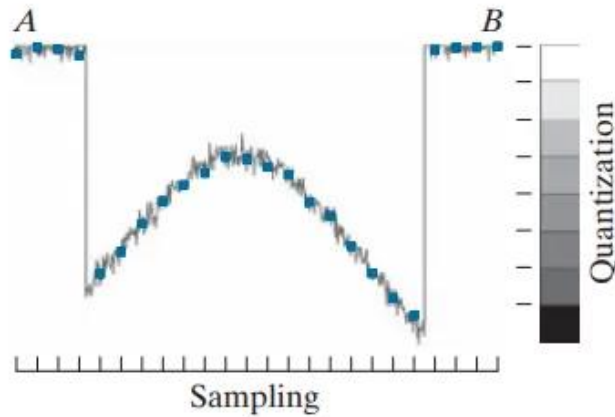
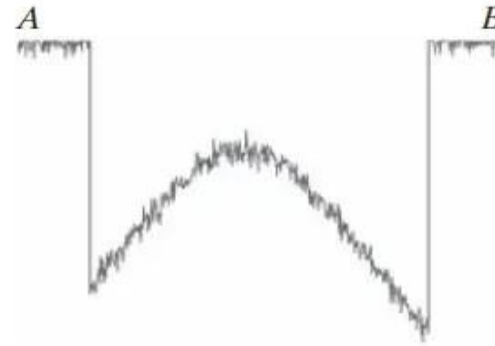
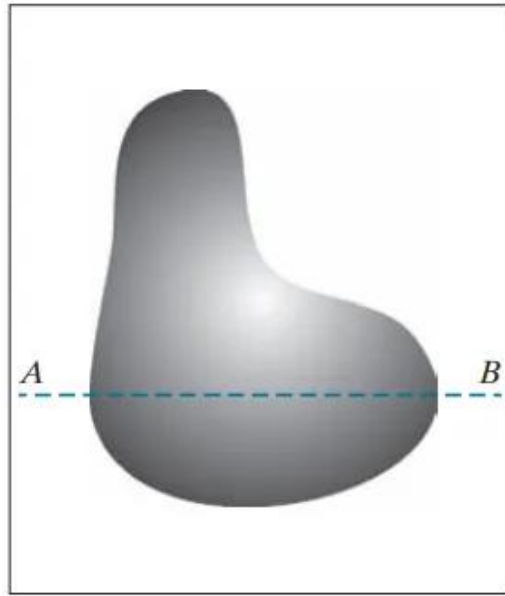
Mammograms



A Typical Image Processing Sequence

Analog to Digital Conversion



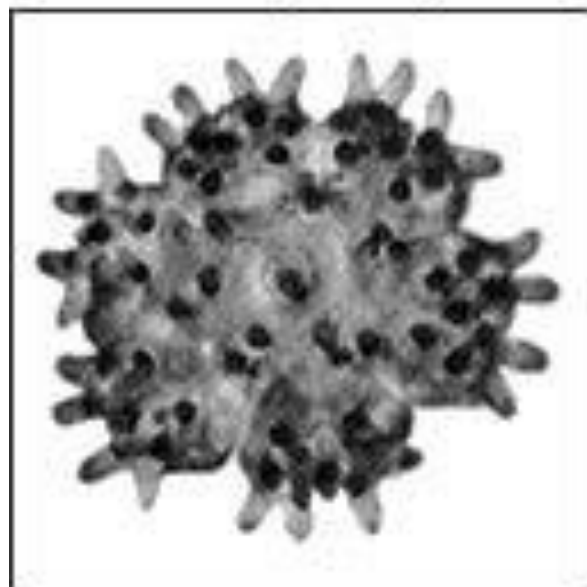


Sampling: the digitization of spatial values x and y .

Quantization: the digitization of amplitude into specific intervals.

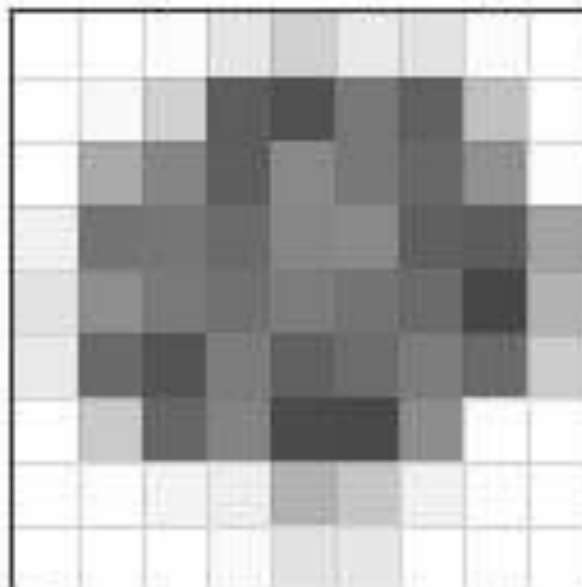
Creation of a Digital Image

Analog Image



(a)

Digital Sampling



(b)

Pixel Quantization

249	244	240	230	209	233	227	251	255
248	245	210	93	81	120	97	193	254
250	170	133	94	137	120	104	145	253
241	116	118	107	134	138	96	92	163
277	142	121	113	124	115	107	71	179
234	106	84	125	97	108	125	106	204
241	202	102	132	75	73	141	246	252
253	252	244	239	178	199	242	250	245
255	249	244	250	226	231	240	251	253

(c)

Figure 1

The Physics of Imaging

- How images are formed
 - Cameras
 - What a camera does
 - How to tell where the camera was
 - Light
 - the measurement of light
 - how much light is coming from the source
 - how much light is reflected by the surface
 - Color
 - Underlying mechanism of color
 - How to describe it and measure it
-

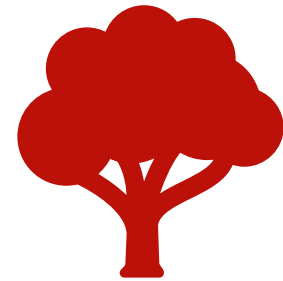
Vision Depends on



Geometry of the Camera



Physics -> The
measurement of the light



Nature of objects in the
world



Lighting affects appearance

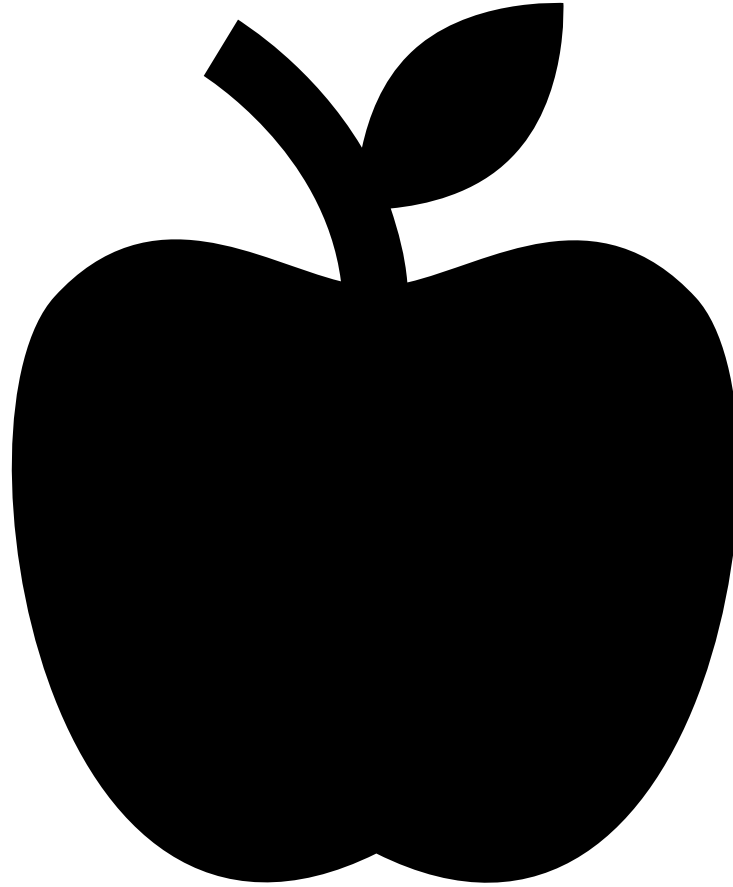


Can Computer match (or beat)
human vision?

- Yes and no

But mostly no!

Example



Where is image processing used?



Machine/Robot
vision



Color
processing



Pattern
recognition



Video
processing



Transmission
and encoding



The medical
field



Image sharpening
and restoring



Facial
recognition

Another Application of Image Processing : Image Fusion



(a)



(b)



(c)

How Machines “See” Images?

What you see 👁️



255	255	218	.	.	.	94	107	127	
251	11	10	8	.	.	.	93	98	120
242	13	11	0	.	.	.	00	08	102
.	19	87	82	78	.	.	.	47	59
.	.	81	78	74	.	.	.	67	77
.	.	60	58	57	.	.	.	82	103
180	.	.	.	.	.	.	.	.	.
219	.	.	.	.	.	.	.	.	.
230	59	.	.	.	.	.	.	.	.
68	.	.	.	.	.	.	.	.	.
69	54	49	52	.	.	.	74	86	82
	60	46	54	.	.	.	77	83	90
	78	74	81	.	.	.	93	101	97

What computers see 🤖

How Machines “See” Images?

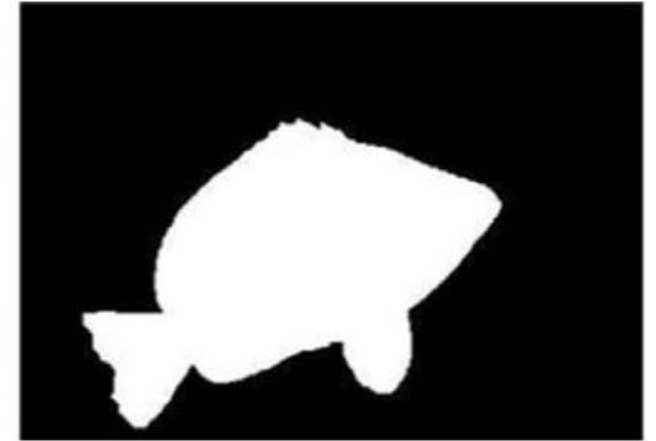
Thus, a computer “sees” digital images as a function: $I(x, y)$ or $I(x, y, z)$, where “ I ” is the pixel intensity and (x, y) or (x, y, z) represent the coordinates (for binary/grayscale or RGB images respectively) of the pixel in the image.



Types of Images

1. Binary Image

Images that have only two unique values of pixel intensity- 0 (representing black) and 1 (representing white) are called binary images.



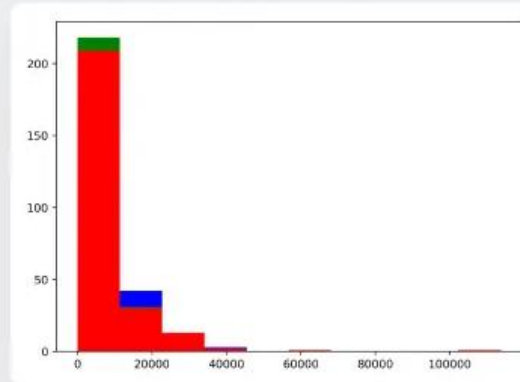
Types of Images

2. Grayscale Image

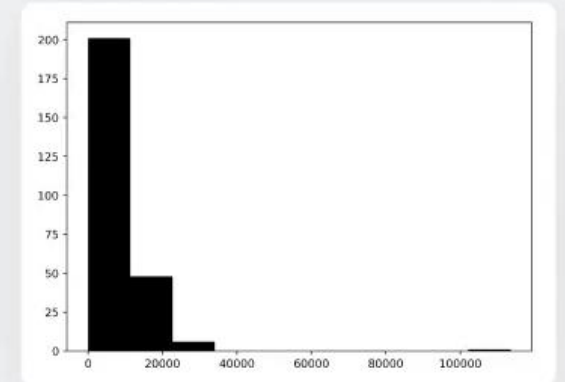
Grayscale or 8-bit images are composed of 256 unique colors, where a pixel intensity of 0 represents the black color and pixel intensity of 255 represents the white color. All the other 254 values in between are the different shades of gray.



Histogram of all RGB Colors



Histogram of Grayscale image



Types of Images

3. RGB Image

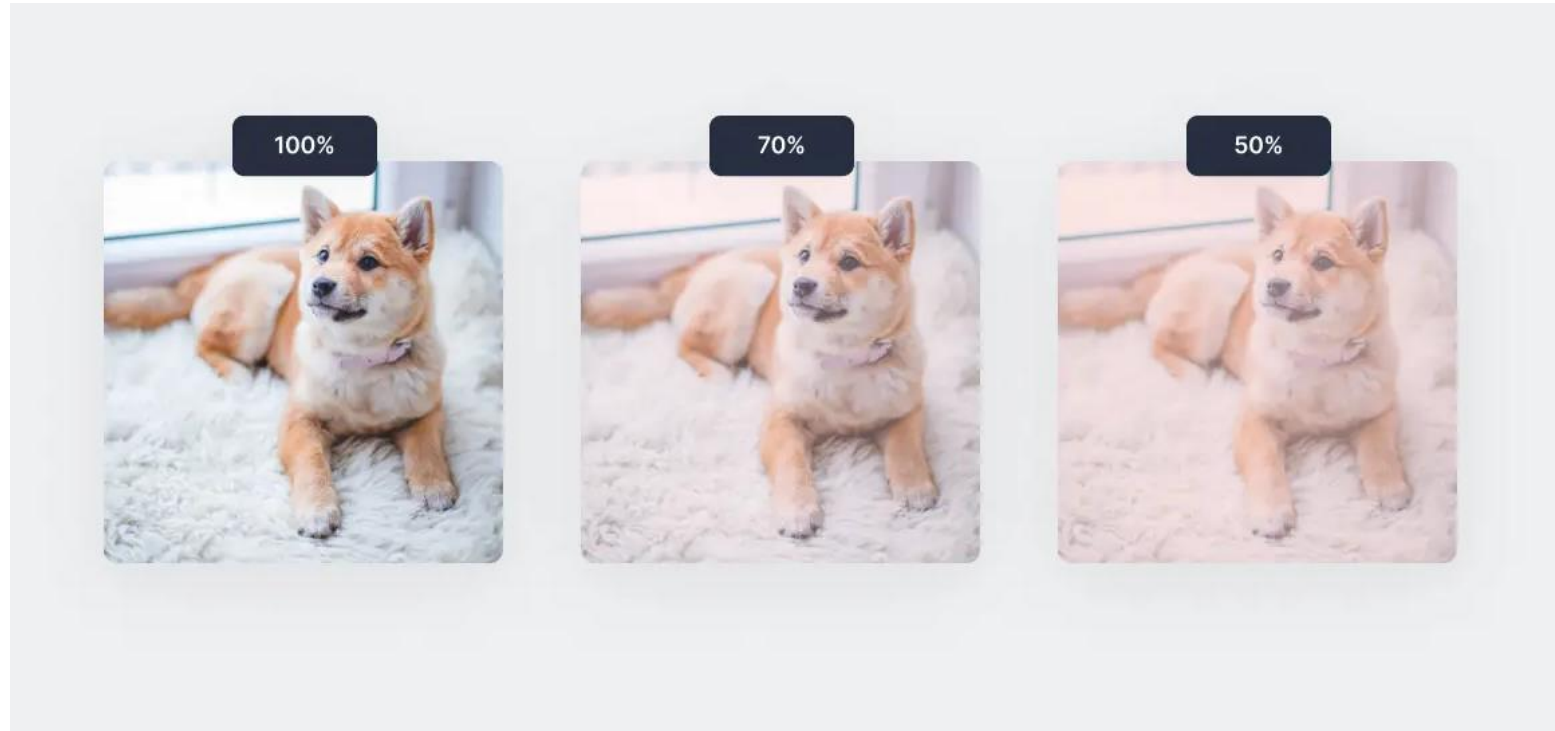
The images we are used to in the modern world are RGB or colored images which are 16-bit matrices to computers. That is, 65,536 different colors are possible for each pixel. “RGB” represents the Red, Green, and Blue “channels” of an image.



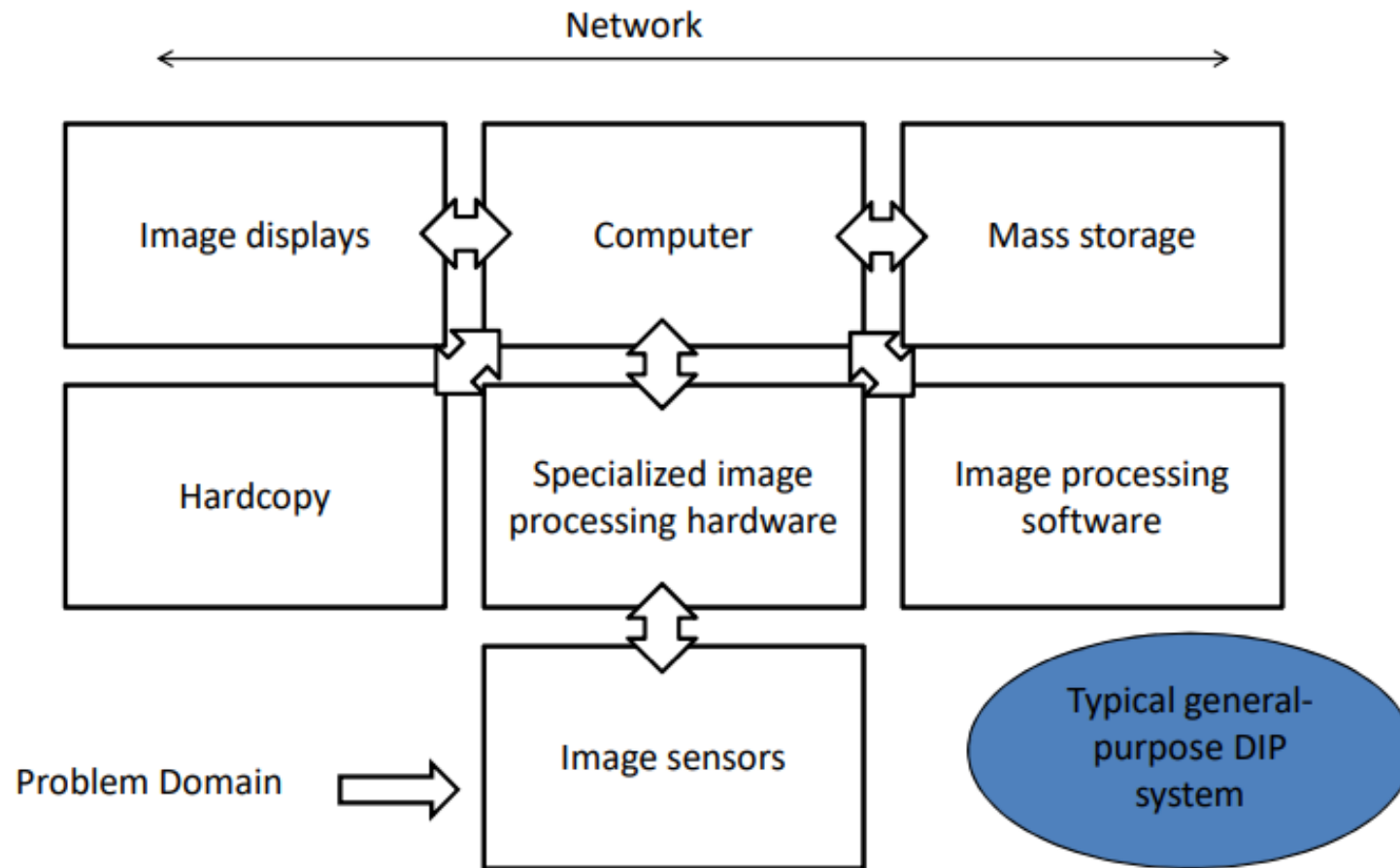
Types of Images

4. RGBA Image

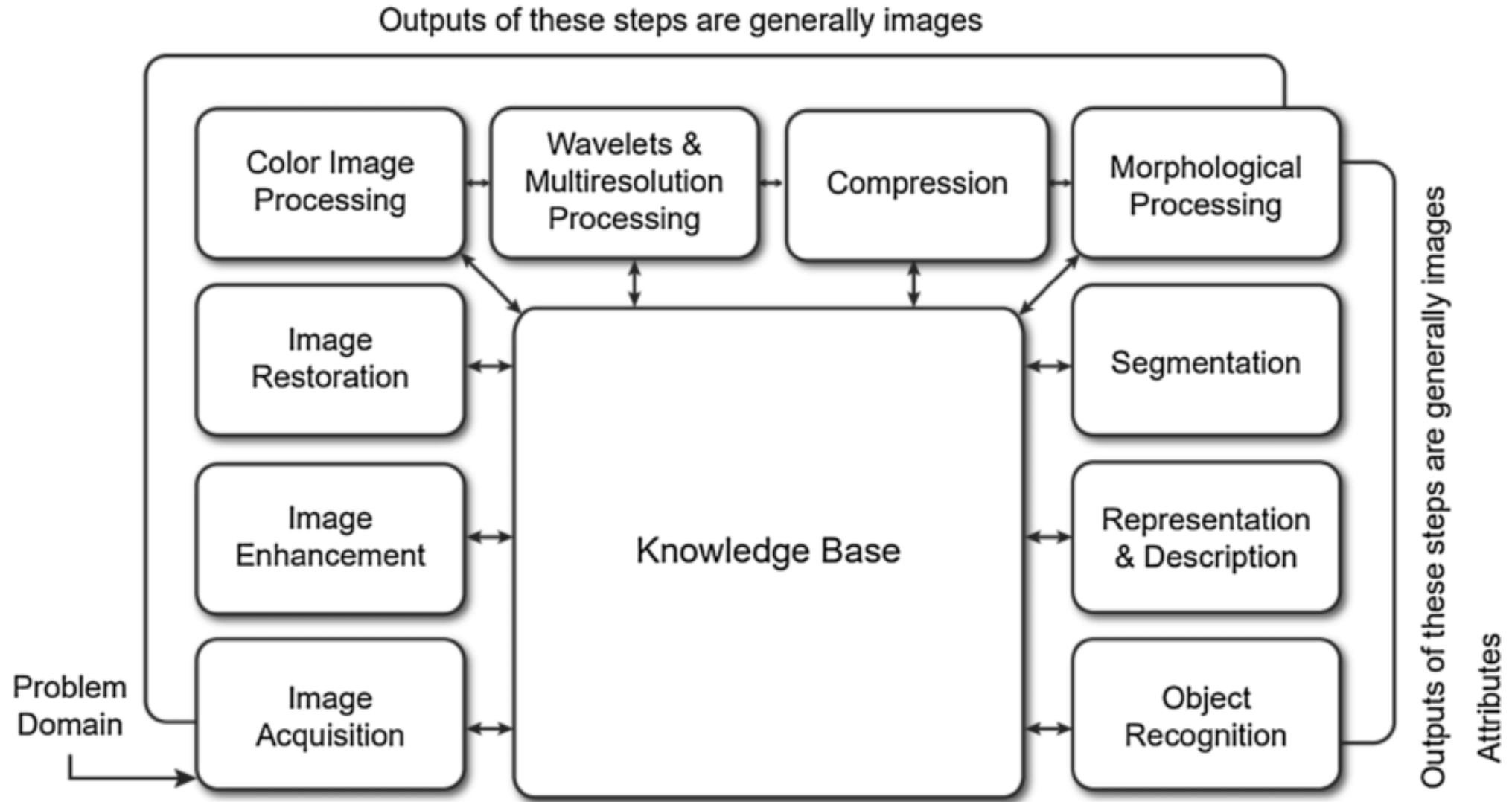
RGBA images are colored RGB images with an extra channel known as “alpha” that depicts the opacity of the RGB image. Opacity ranges from a value of 0% to 100% and is essentially a “see-through” property.



Components of an Image Processing System



Fundamental Steps in Digital Image Processing



Acquisition

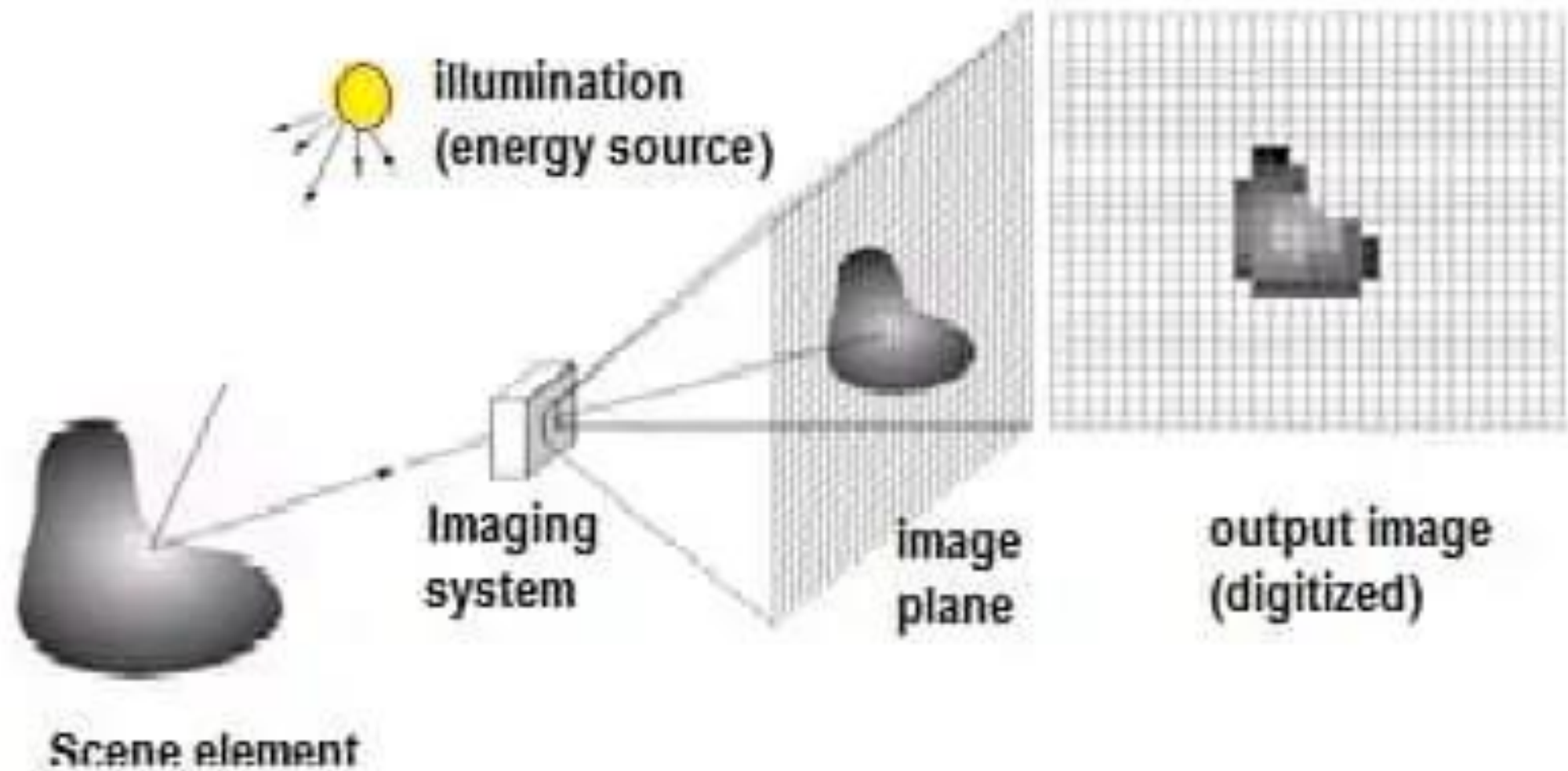


Image Enhancement



Input



Output

Image Restoration



Color Image Processing

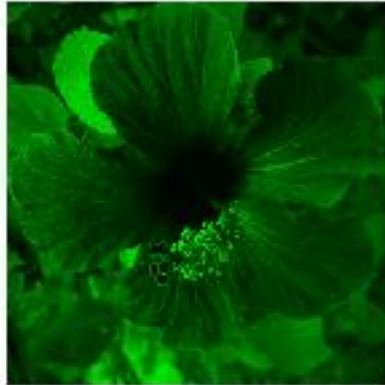
f



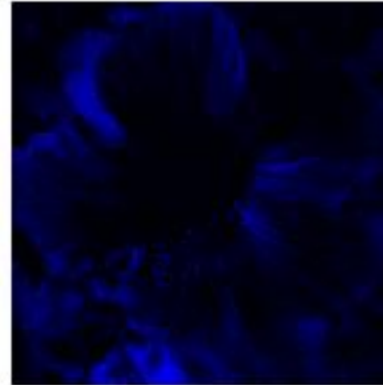
R (Red)



G (green)



B (blue)



Wavelets and Multi-Resolution Processing



Compression

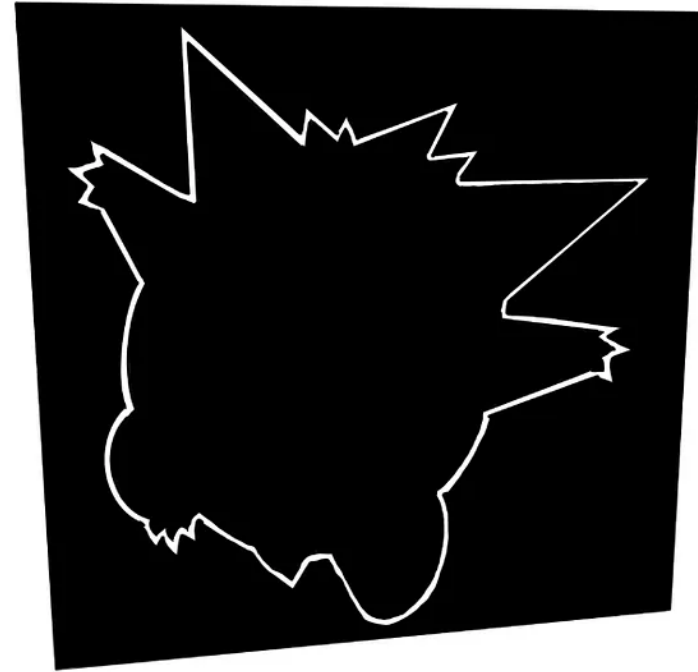


SMART COMPRESSION:
✓ QUALITY PRESERVED



OVERCOMPRESSED:
✗ IMPACTED QUALITY

Morphological Image Processing

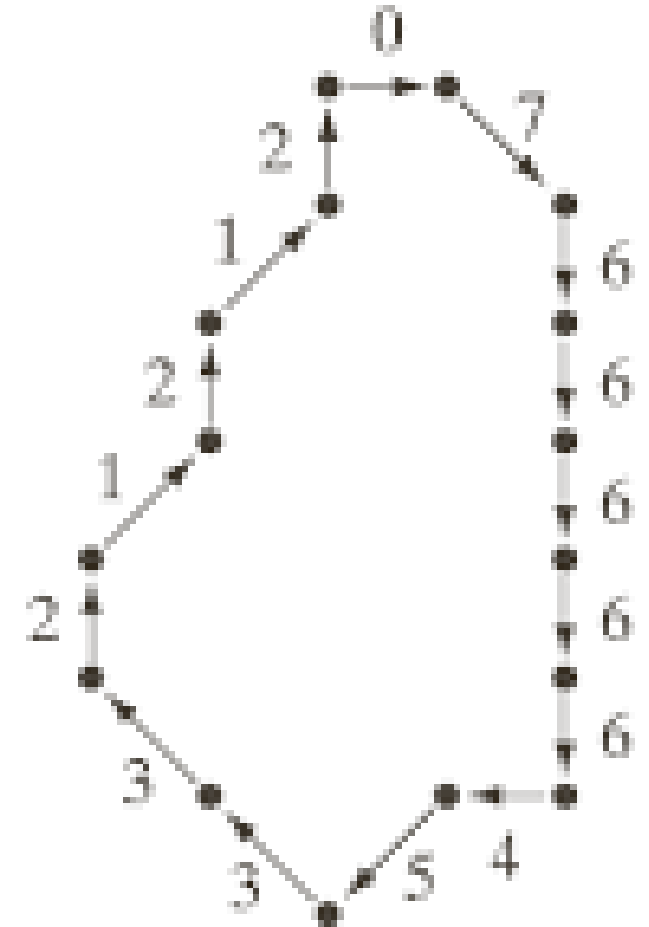
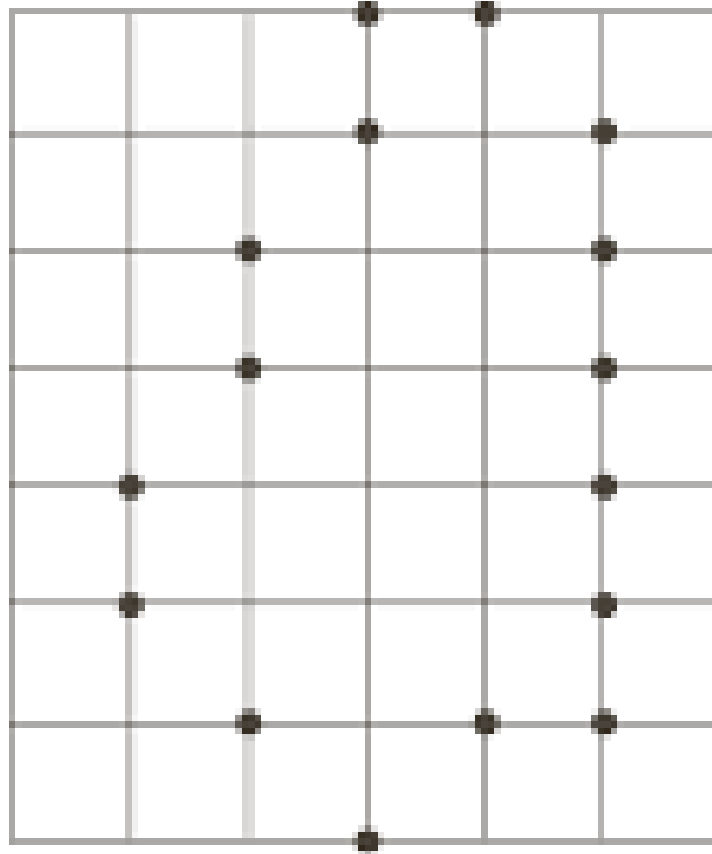
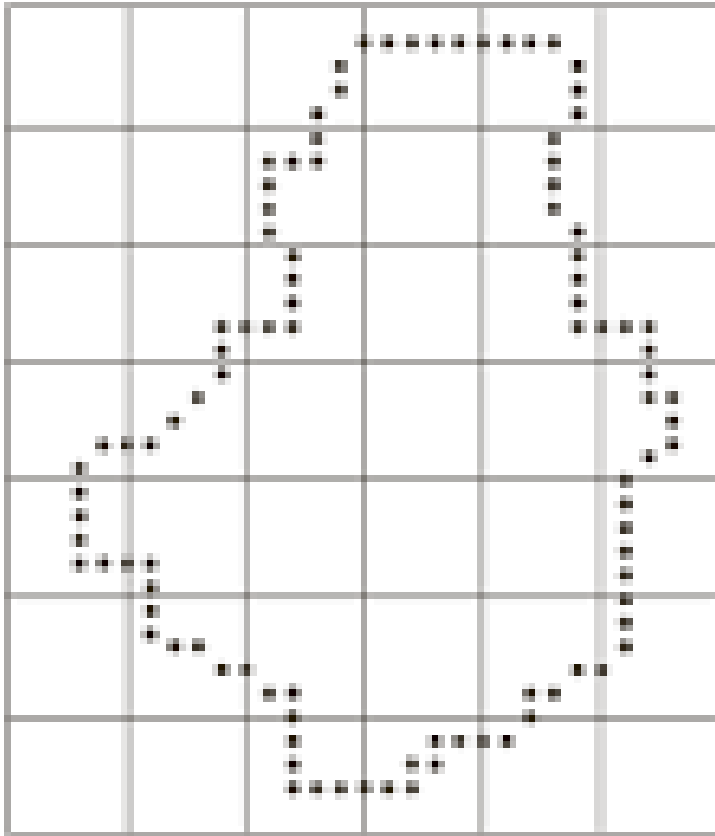


Dilation / Erosion ?

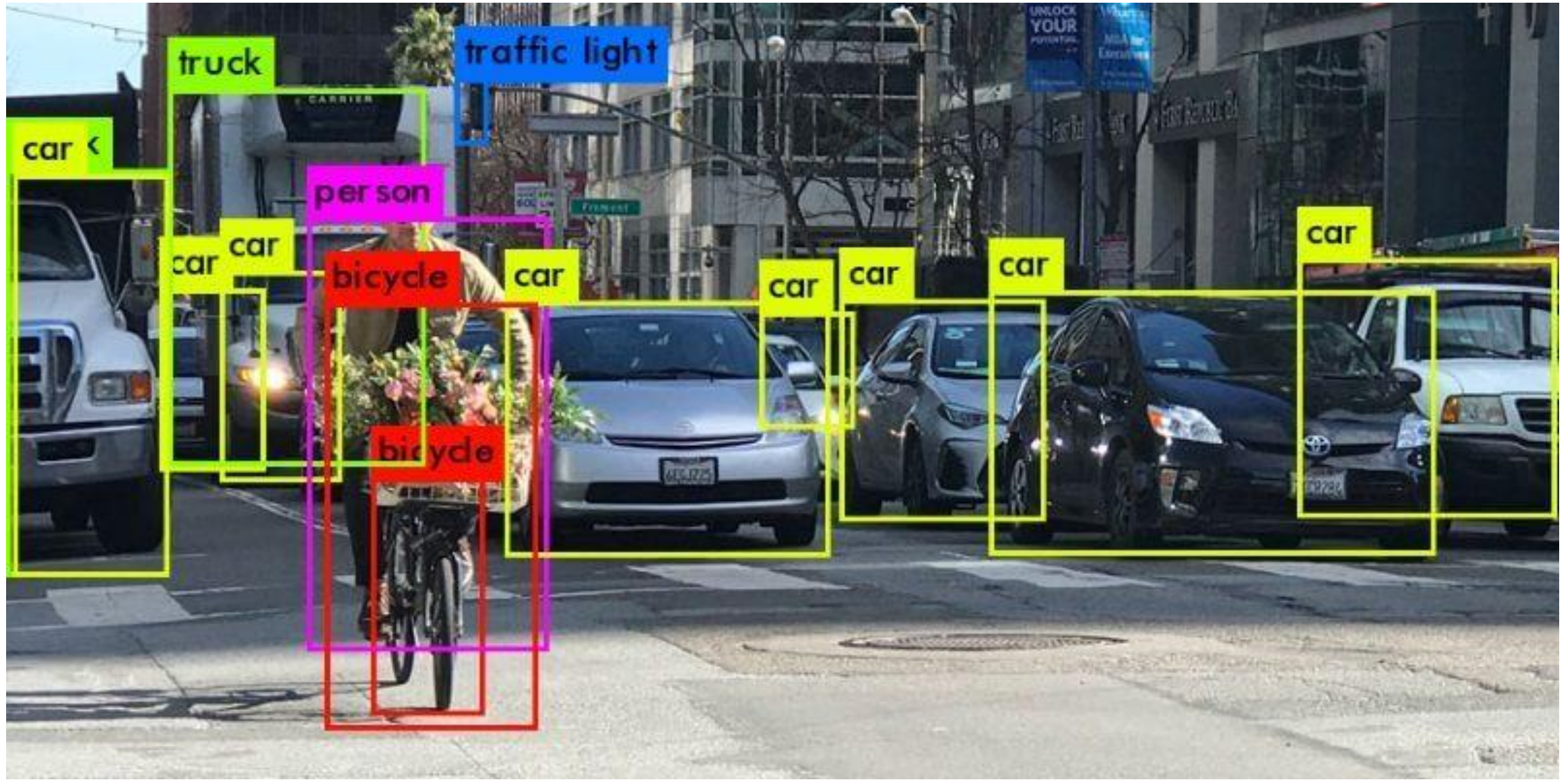
Segmentation



Representation and Description



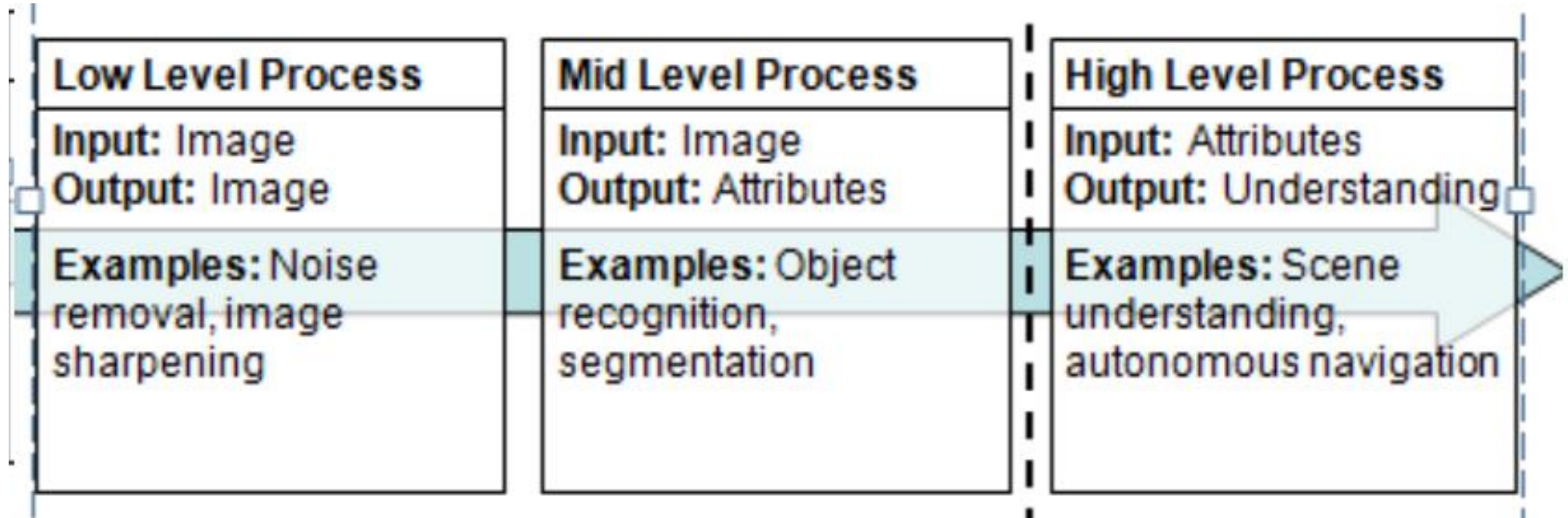
Object Recognition



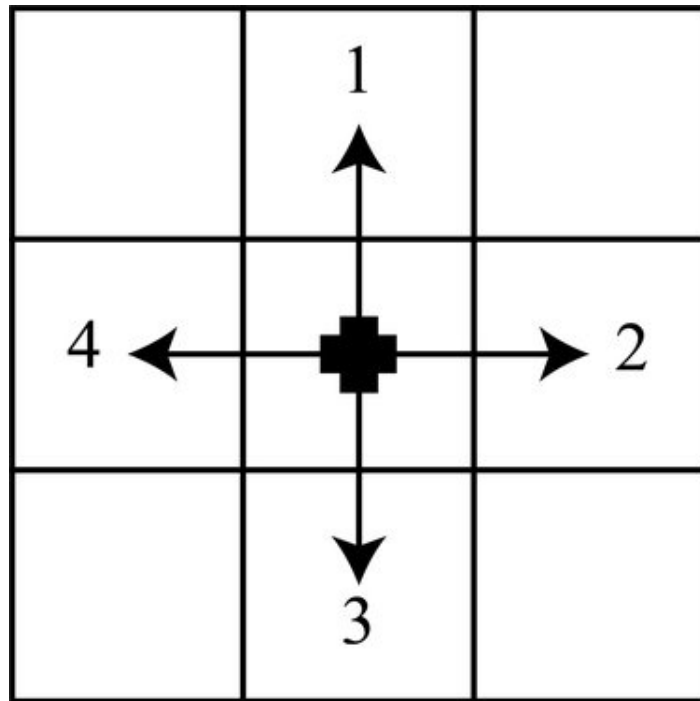
Knowledge Base

- Knowledge may be as simple as the bounding box coordinates for an object of interest that has been found in the image, along with the object label assigned to it.
 - Anything that will help in solving the problem for the specific task at hand can be encoded into the knowledge base.
-

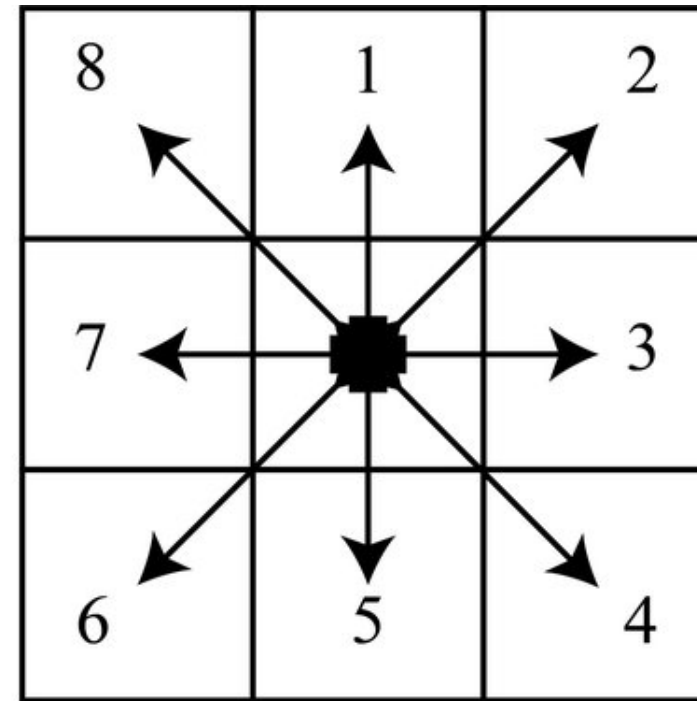
3 levels of Image Processing



Relationship between Pixels (Neighbourhood of a Pixel)

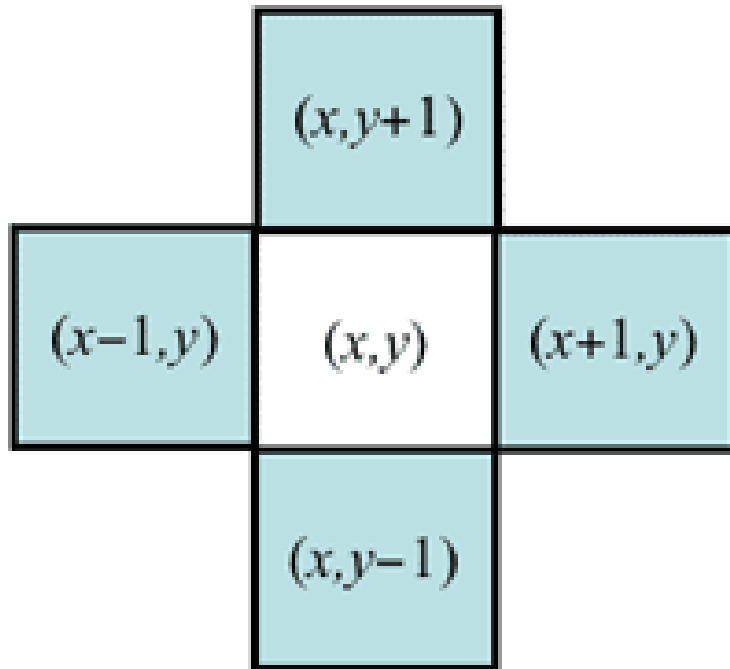


4-Connected

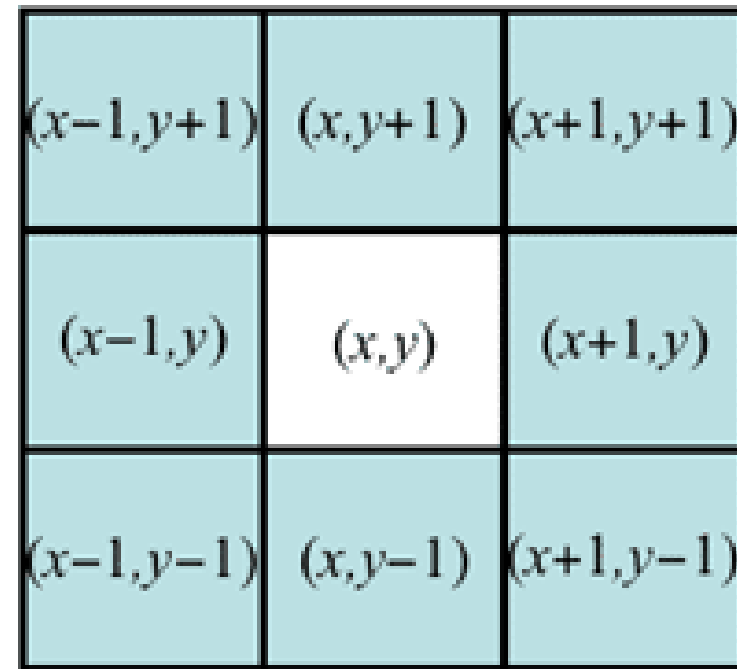


8-Connected

Relationship between Pixels (Neighbourhood of a Pixel)



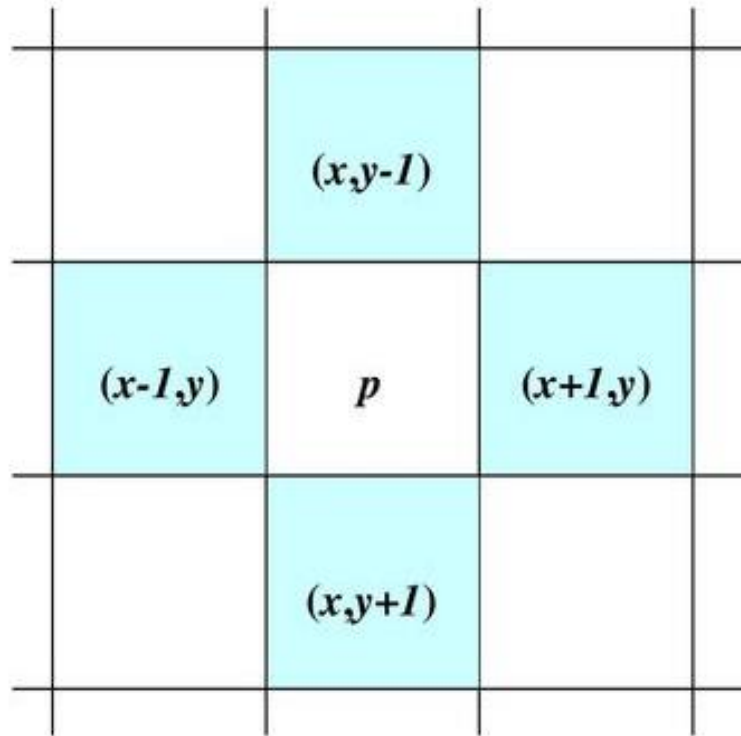
4-neighbourhood



8-neighbourhood

Relationship between Pixels

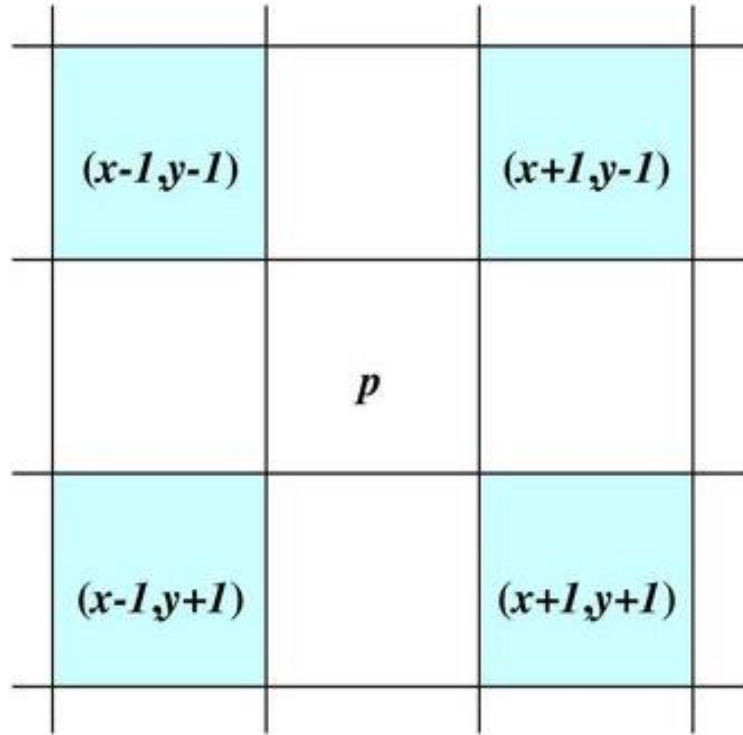
Neighborhood relation is used to tell adjacent pixels. It is useful for analyzing regions.



4-neighbors of p :

$$N_4(p) = \left\{ \begin{array}{l} (x-1, y) \\ (x+1, y) \\ (x, y-1) \\ (x, y+1) \end{array} \right\}$$

Relationship between Pixels



Diagonal neighbors of p :

$$N_D(p) = \left\{ \begin{array}{l} (x-1, y-1) \\ (x+1, y-1) \\ (x-1, y+1) \\ (x+1, y+1) \end{array} \right\}$$

Relationship between Pixels

$(x-1, y-1)$	$(x, y-1)$	$(x+1, y-1)$
$(x-1, y)$	p	$(x+1, y)$
$(x-1, y+1)$	$(x, y+1)$	$(x+1, y+1)$

8-neighbors of p :

$$N_8(p) = \left\{ \begin{array}{l} (x-1, y-1) \\ (x, y-1) \\ (x+1, y-1) \\ (x-1, y) \\ (x+1, y) \\ (x-1, y+1) \\ (x, y+1) \\ (x+1, y+1) \end{array} \right\}$$

8-neighborhood relation considers all neighbor pixels.

Adjacency between pixels

Let V be the set of intensity values used to define adjacency.

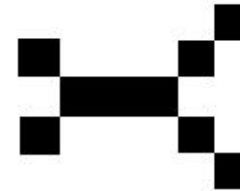
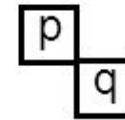
- Three types of Adjacency:
 - 4- Adjacency
 - 8- Adjacency
 - m-Adjacency
-

Adjacency between pixels

- **4-adjacency:** $q \in N_4(p)$



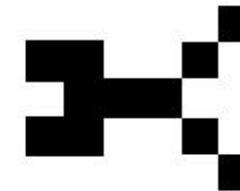
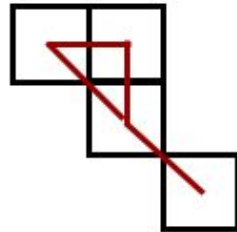
- **8-adjacency:** $q \in N_8(p)$



- **m-adjacency:** $q \in N_4(p)$ or $q \in N_D(p)$ and $N_4(p) \cap N_4(q) = \emptyset$

Example:

8



Connectivity between pixels

0 1 1
0 1 0
0 0 1

Fig: An arrangement
of pixels

0 1—1
0 1 0
0 0 1

Fig: 4-connectivity of
pixels

0 1—1
0 1 0
0 0 1

Fig: 8-connectivity of
pixels

0 1—1
0 1 0
0 0 1

Fig: m-connectivity of
pixels

m-adjacency (mixed adjacency)

- If the pixel can be connected with the 4 adjacency and as well as with the 8 adjacency, then we the priority is given to 4 adjacency.

0	1	1
0	1	0
0	0	1

0	1	1
0	1	0
0	0	1

0	1	1
0	1	0
0	0	1

Example of Binary Image

• $V = \{1\}$

0	1	0	1
0	0	1	0
0	0	1	0
1	0	0	0

a) 4 adjacency

b) 8 adjacency

Example of Binary Image

- $V = \{1\}$

0	1	0	1
0	0	1	0
0	0	1	0
1	0	0	0

a) 4 adjacency

Example of Binary Image

- $V = \{1\}$

0	1	0	1
0	0	1	0
0	0	1	0
1	0	0	0

The diagram shows a 4x4 grid of binary values. The central pixel at row 1, column 2 (0-indexed) has a value of 1. Red lines connect this central pixel to its eight neighbors: (0,1), (0,3), (2,1), and (2,3). The other pixels have values 0 or 1, but only the central pixel is connected by red lines.

b) 8 adjacency

Example of Binary Image

- $V = \{1\}$

0	1	0	1
0	0	1	0
0	0	1	0
1	0	0	0

Not adjacent

b) 8 adjacency

Example of Grayscale Image

- $V = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

54	10	100	8
81	150	2	34
201	203	3	45
7	70	147	56

a) 4 adjacency

b) 8 adjacency

Example of Grayscale Image

- $V = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

a) 4 adjacency

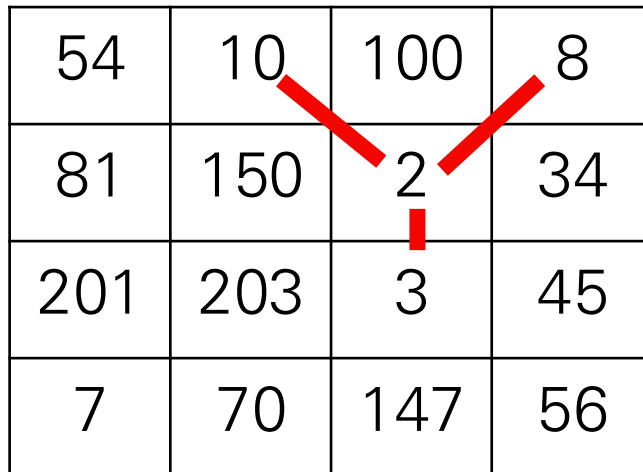
54	10	100	8
81	150	2	34
201	203	3	45
7	70	147	56

Example of Grayscale Image

- $V = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

b) 8 adjacency

54	10	100	8
81	150	2	34
201	203	3	45
7	70	147	56



Example of Grayscale Image

- $V = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

b) 8 adjacency

54	10	100	8
81	150	2	34
201	203	3	45
7	70	147	56

Not adjacent

Why do we check relationship between pixels?

1. To Detect Edges and Boundaries

- Pixels belonging to the same object usually have similar color or brightness values.
- By checking pixel relationships, algorithms can find where values change sharply.
- This allows the computer to trace outlines and boundaries around objects.

2. To Separate Objects (Segmentation)

- Connecting pixels with similar properties helps the computer isolate specific objects.
 - For example, it allows a medical AI to separate a tumor from healthy tissue in an MRI scan.
 - It is also how self-driving cars separate the road from pedestrians.
-

Why do we check relationship between pixels?

3. To Measure Size and Distance

- Distance metrics (like City-Block or Chessboard distance) help calculate the true size of an object.
- This allows computers to measure the area, perimeter, and length of shapes within an image.

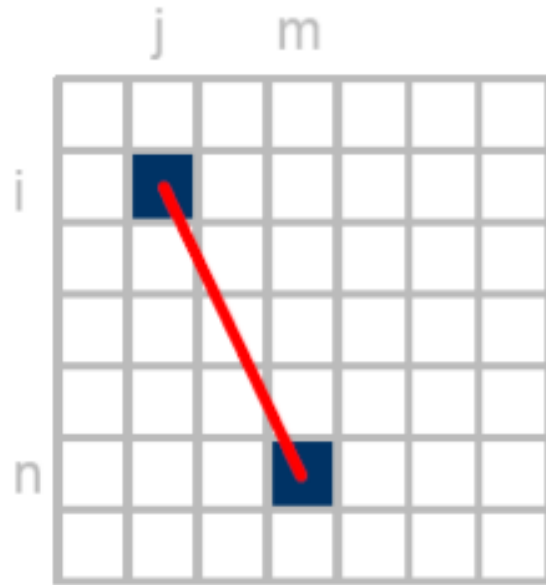
4. To Clean Up Image Noise

- If a single pixel is vastly different from all its 8-neighbors, it is usually "noise" (a glitch or speckle).
- Checking this relationship allows filters to smooth out the image by changing that pixel to match its surroundings.

5. To Track Motion

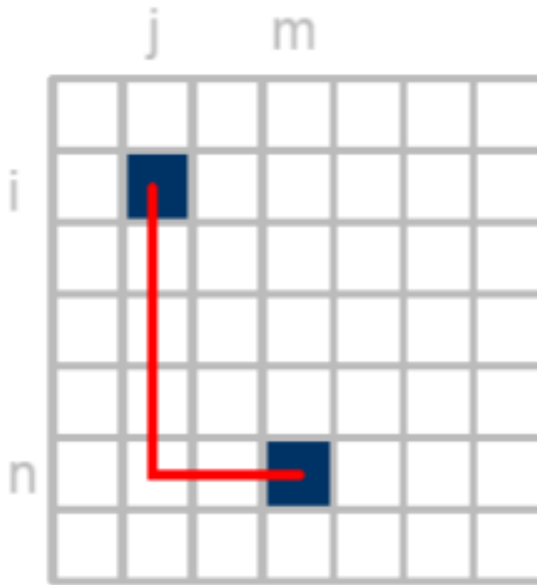
- In videos, algorithms look at how a group of connected pixels moves from one frame to the next.
 - This relationship is critical for video surveillance, gesture controls, and video compression.
-

Distance Measures between Pixels



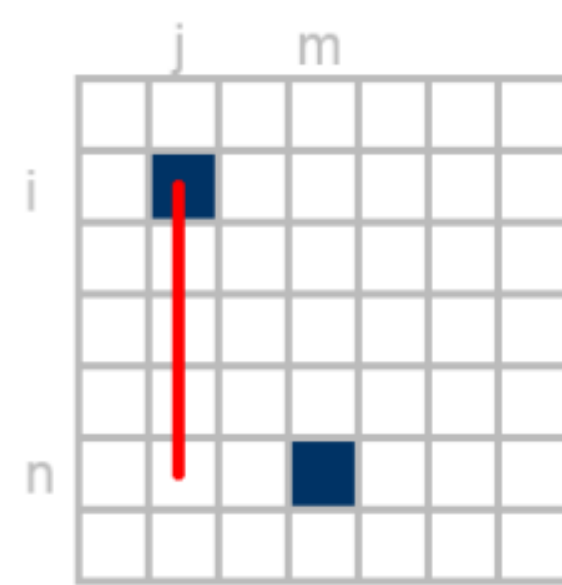
Euclidean Distance

$$= \sqrt{(i-n)^2 + (j-m)^2}$$



City Block Distance

$$= |i-n| + |j-m|$$



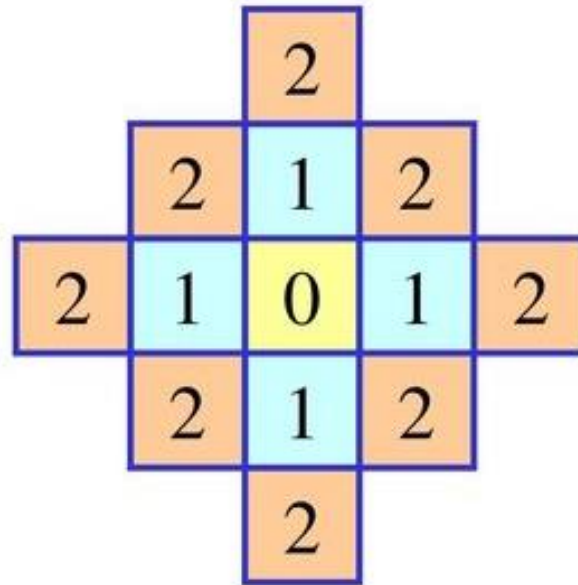
Chessboard Distance

$$= \max[|i-n|, |j-m|]$$

Distance Measures between Pixels

D_4 -distance (city-block distance) is defined as

$$D_4(p, q) = |x - s| + |y - t|$$



Distance Measures between Pixels

D_8 -distance (chessboard distance) is defined as

$$D_8(p, q) = \max(|x - s|, |y - t|)$$

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

D_m Distance

- is defined as the shortest m-path between the points.
 - In this case, the distance between two pixels will depend on the values of the pixels along the path, as well as the values of their neighbors.
-

Example

- Compute the distance between the two pixels using the three distances :
- $q:(1,1)$
- $p:(2,2)$

	1	2	3
1	q		
2		p	
3			

Example

- Euclidian distance :

$$((1-2)^2 + (1-2)^2)^{1/2} = \text{sqrt}(2).$$

- D4(City Block distance):

$$|1-2| + |1-2| = 2$$

- D8(chessboard distance) :

$$\max(|1-2|, |1-2|) = 1$$

	1	2	3
1	q		
2		p	
3			

Example : Use the city block distance to prove 4-neighbors ?

- Pixel A : $|2-2| + |1-2| = 1$
- Pixel B: $|3-2| + |2-2| = 1$
- Pixel C: $|2-2| + |2-3| = 1$
- Pixel D: $|1-2| + |2-2| = 1$

	1	2	3
1		d	
2	a	p	c
3		b	

Example : Use the chessboard distance to prove 8-neighbors ?

- Pixel A : $\max(|2-2|, |1-2|) = 1$
- Pixel B: $\max(|3-2|, |2-2|) = 1$
- Pixel C: $\max(|2-2|, |3-2|) = 1$
- Pixel D: $\max(|1-2|, |2-2|) = 1$
- Pixel E : $\max(|1-2|, |1-2|) = 1$
- Pixel F: $\max(|1-2|, |3-2|) = 1$
- Pixel G: $\max(|3-2|, |1-2|) = 1$
- Pixel H: $\max(|3-2|, |3-2|) = 1$

	1	2	3
1	e	d	f
2	a	p	c
3	g	b	h

Example 1

- An image segment is shown below. Let V be the set of gray level values used to define connectivity in the image. Compute D_4 , D_8 and D_m distances between pixels 'p' and 'q' for:

- $V = \{2, 3\}$

- $V = \{2, 6\}$

2 p	3	2	6	1
6	2	3	6	2
5	3	2	3	5
2	4	3	5	2
4	5	2	3	6 q

Example 1

- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 4,4$

$$\begin{aligned} D4 &= |x-s| + |y-t| \\ &= |0-4| + |0-4| \\ &= 4 + 4 = 8 \text{ units} \end{aligned}$$

2 p	3	2	6	1
6	2	3	6	2
5	3	2	3	5
2	4	3	5	2
4	5	2	3	6 q

Example 1

- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 4,4$

$$\begin{aligned} D8 &= \max(|x-s|, |y-t|) \\ &= \max(|0-4|, |0-4|) \\ &= \max(4, 4) = 4 \text{ units} \end{aligned}$$

2 p	3	2	6	1
6	2	3	6	2
5	3	2	3	5
2	4	3	5	2
4	5	2	3	6 q

Example 1

- $V = \{2,3\}$
- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 4,4$

$D_m = A$ As $q(6)$ is not included in the set V ,
so there is no path between p and q .

2 p	3	2	6	1
6	2	3	6	2
5	3	2	3	5
2	4	3	5	2
4	5	2	3	6 q

- Let's take $V = \{2,6\}$
-

Example 1

- $V = \{2,6\}$
- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 4,4$

Dm = There is no path between p and q.

2 ^p	3	2	6	1
6	2	3	6	2
5	3	2	3	5
2	4	3	5	2
4	5	2	3	6 ^q

Example 2

- Consider the following image segment. Compute D4, D8 and Dm distances between pixels 'p' and 'q' for:
 - $V = \{0,1\}$
 - $V = \{1,2\}$

3	1	2	1 (q)
0	2	0	2
1	2	1	1
1 (p)	0	1	2

Example 2

- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 3,3$

$$\begin{aligned} D4 &= |x-s| + |y-t| \\ &= |0-3| + |0-3| \\ &= 3 + 3 = 6 \text{ units} \end{aligned}$$

	3	1	2	1 (q)	3,3
	0	2	0	2	
	1	2	1	1	
0,0	1 (p)	0	1	2	

Example 2

- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 3,3$

$$\begin{aligned} D8 &= \max(|x-s|, |y-t|) \\ &= \max(|0-3|, |0-3|) \\ &= \max(3, 3) = 3 \text{ units} \end{aligned}$$

	3	1	2	1 (q)	3,3
	0	2	0	2	
	1	2	1	1	
0,0	1 (p)	0	1	2	

Example 2

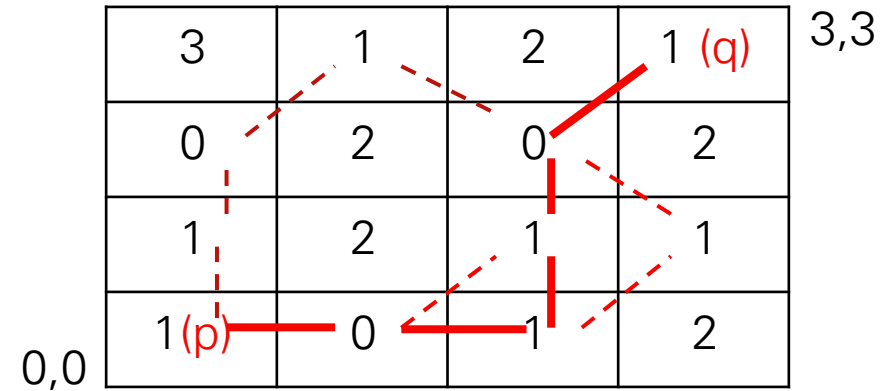
- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 3,3$
- $V = \{0,1\}$

$D_m =$

	3	1	2	1 (q)	3,3
	0	2	0	2	
	1	2	1	1	
0,0	1 (p)	0	1	2	

Example 2

- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 3,3$
- $V = \{0,1\}$
 - $D_m = 5$ Units



Example 2

- Coordinates of $p(x,y) = 0,0$
- Coordinates of $q(s,t) = 3,3$
- $V = \{1,2\}$

$D_m = 6$ Units

3x	1	2	1 (q)	3,3
0	2	0	2	
1	2	1	1	
0,0	1 (p)	0	1	2

Quiz 1

- Look at this small pixel pattern:

1	0
0	1

- If your computer algorithm is strictly using 4-connectivity (up, down, left, right), how many distinct objects will it count here
A) 1 object B) 2 objects C) 0 objects

Option B - 2 objects

Quiz 2

- Imagine you want to move a chess King from pixel coordinate (2, 2) to pixel coordinate (5, 5). The King can move in any direction (up, down, left, right, or diagonal). What is the Chessboard Distance (D_8) between these two pixels?

A) 6 B) 3 C) 5

Option B - 3

Quiz 3

You are building an algorithm to clean up digital camera noise. You find a single bright white pixel. All eight pixels surrounding it are completely black. What is the most likely pixel relationship here?

- A) The white pixel is part of a large object border.
- B) The white pixel is an isolated "noise" glitch.
- C) The pixels are 4-adjacent.

Option B - The white pixel is an isolated "noise" glitch.

Arithmetic Operations between Images

- Addition $\rightarrow f(x) + g(x)$
- Subtraction $\rightarrow f(x) - g(x)$
- Multiplication $\rightarrow f(x) * g(x)$
- Division $\rightarrow f(x) / g(x)$

Rule

1. If the result is floating point number, round of its value.
 2. If the result is above the pixel range, exceeds 255, select the max range.
 3. If the result is below the pixel range, select the min range.
 4. If the result is infinity then consider 0.
-

Addition

0	100	10
4	0	10
8	0	5

 $+$

10	100	5
2	0	0
0	10	10

 $=$

10	200	15
6	0	10
8	10	15

Uses

1. Addition of noisy images for noise reduction.
 2. Image averaging in the field of astronomy
-

Subtraction

0	100	10
4	0	10
8	0	5

 -

10	100	5
2	0	0
0	10	10

 =

0	0	5
2	0	10
8	0	0

Uses

1. Enhancement of differences between images.
 2. Mask mode radiography in medical imaging.
-

Multiplication

0	100	10
4	0	10
8	0	5

 $*$

10	100	5
2	0	0
0	10	10

 $=$

0	255	50
8	0	0
0	0	50

Uses

1. Shading correction.
 2. Masking or Region of Interest (ROI) operations.
-

Division

0	100	10
4	0	10
8	0	5

*

10	100	5
2	0	0
0	10	10

=

0	1	2
2	0	0
0	0	1

Uses

1. Shading correction.
-

Logical Operations Between Images

- AND
- OR
- NOT

1	1	0
0	0	1
1	0	1

AND

0	1	0
1	1	0
0	0	1

=

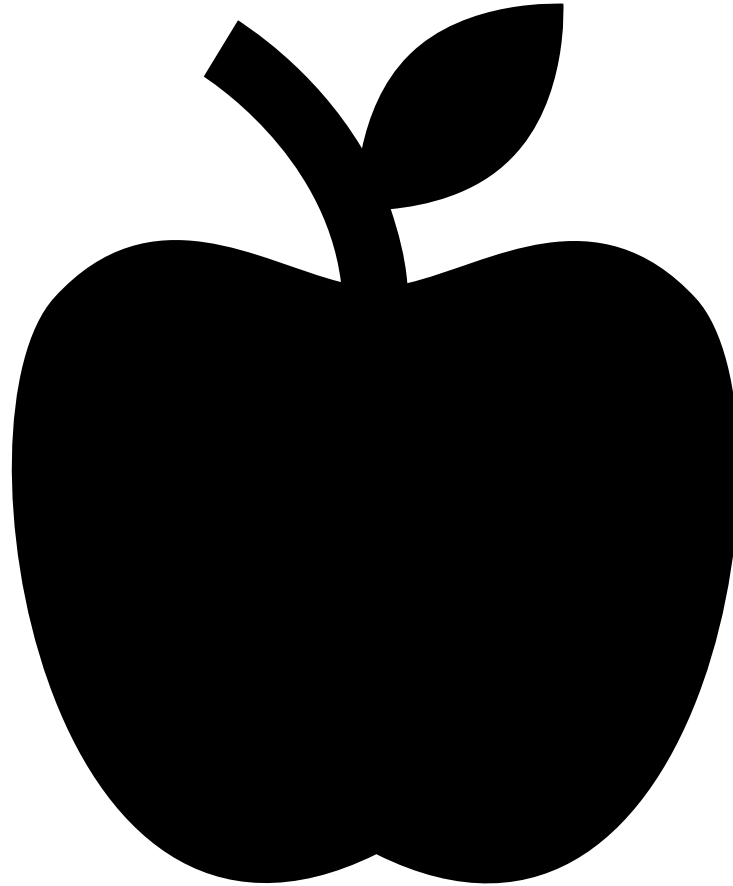
0	1	0
0	0	0
0	0	1

Logical Operations Between Images

1	1	0	OR	0	1	0	=	1	1	0
0	0	1		1	1	0		1	1	1
1	0	1		0	0	1		1	0	1

NOT	1	1	0	=	0	0	1
	0	0	1		1	1	0
	1	0	1		0	1	0

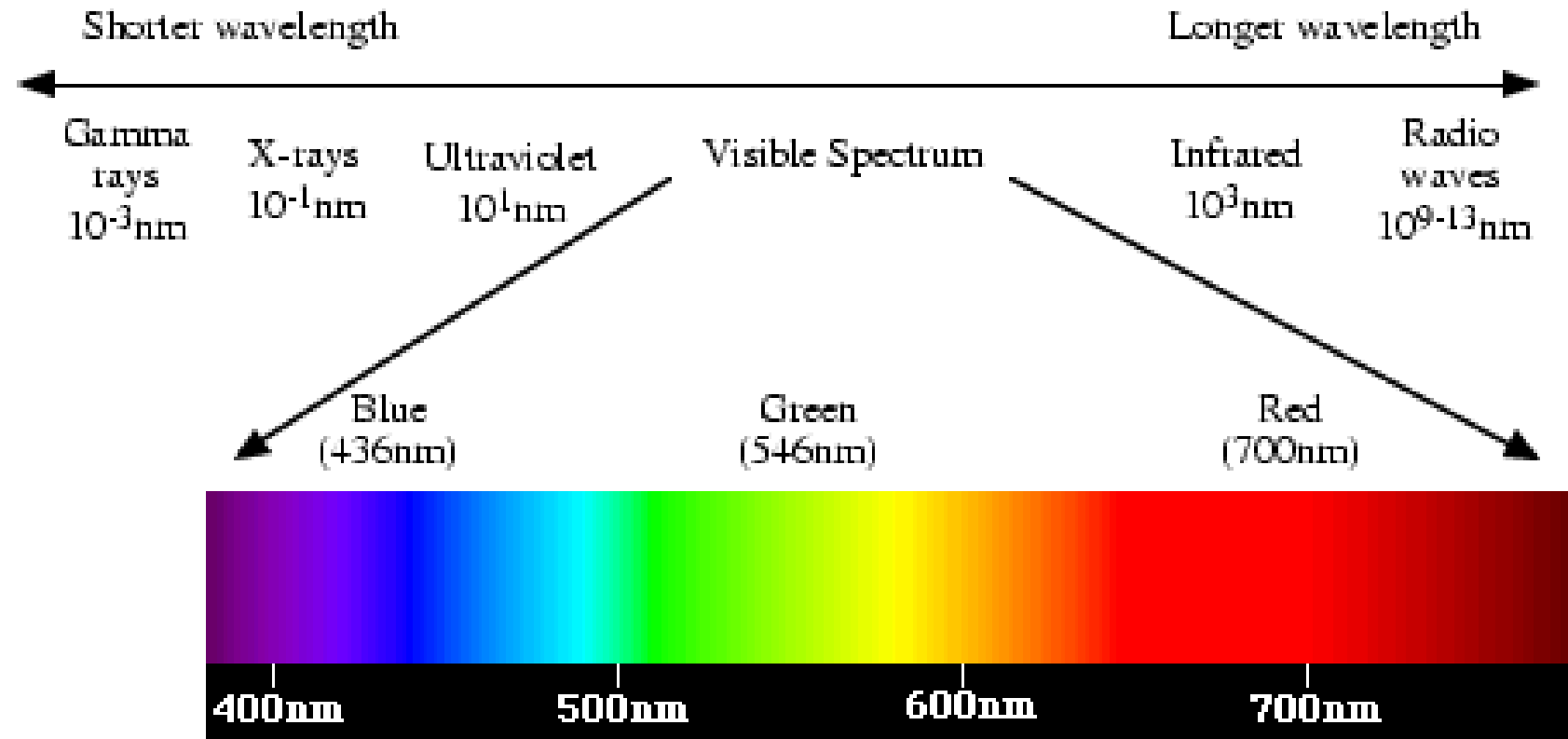
Color Fundamentals



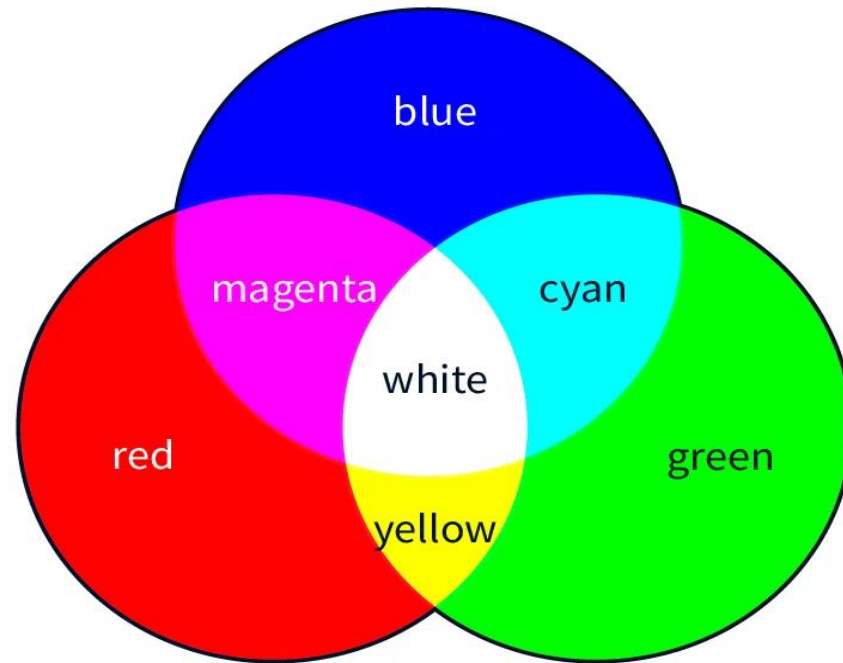


Color Image Processing

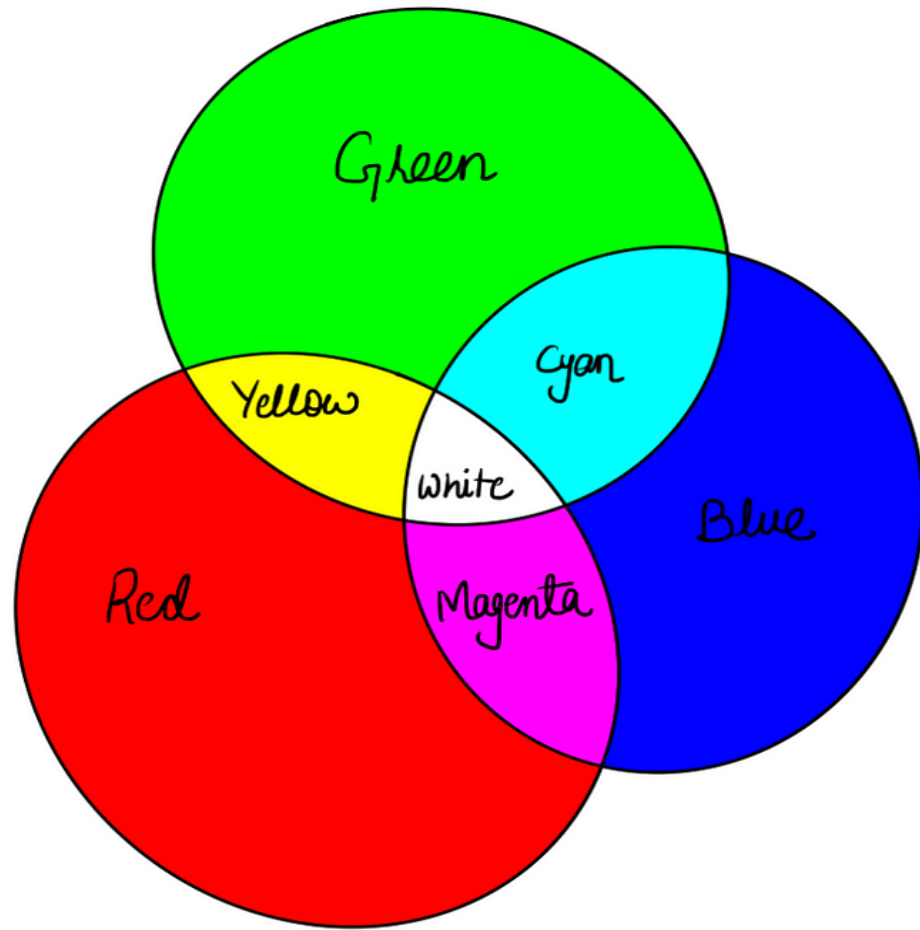
Color Image Processing



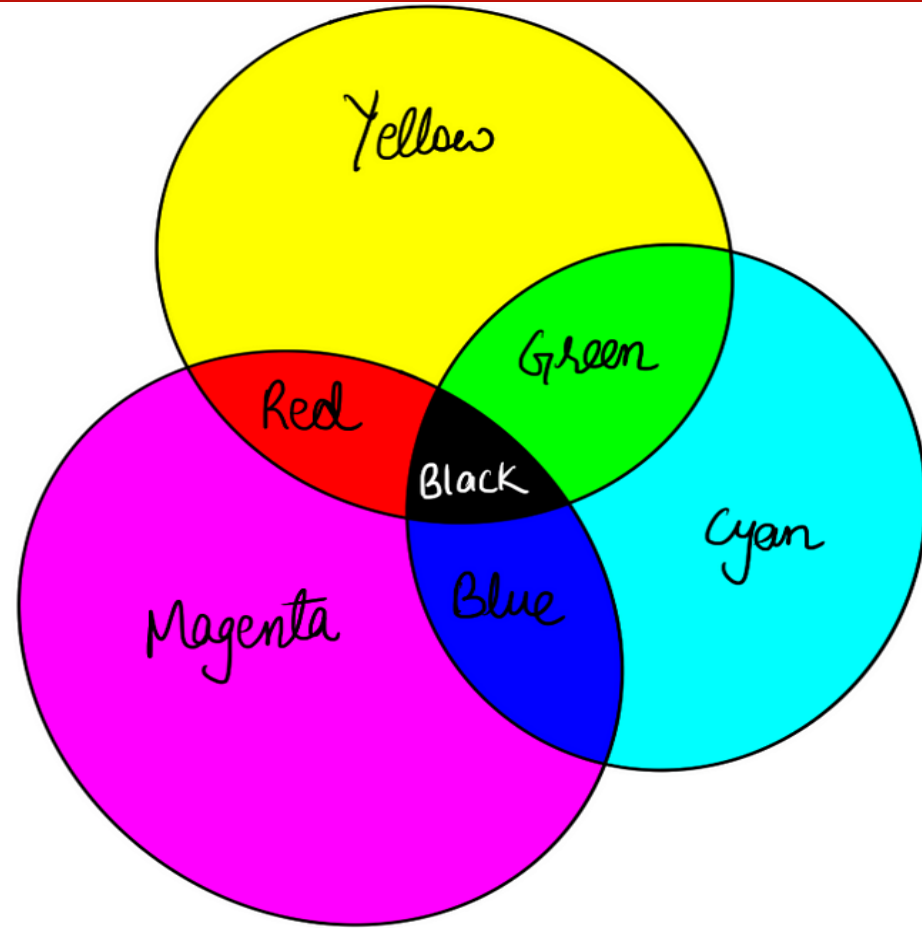
RGB Model



CMY Model



**Mixture of Light
(Additive Primaries)**



**Mixture of Pigments
(Subtractive Primaries)**

CMY Model

- $B = (0,0,0)$ $W = (1,1,1)$
- Conversion from RGB to CMY

$$\begin{array}{ccc} C & & R \\ M = 1 - & G & \\ Y & & B \end{array}$$

- Conversion from CMY to RGB

$$\begin{array}{ccc} R & & C \\ G = 1 - & M & \\ B & & Y \end{array}$$

RGB to CMY Conversion with an example

- RGB (98,35, 146) To CMY

- 1st Step – NORMALIZATION

- $R = 98/255 = 0.384$

- $G = 35/255 = 0.137$

- $B = 146/255 = 0.572$

- 2nd Step – TO CMY

- $C = 1 - 0.384 = 0.616$

- $M = 1 - 0.137 = 0.863$

- $Y = 1 - 0.572 = 0.428$

HSI Model



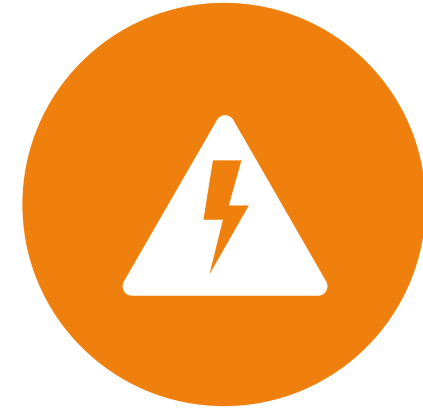
Hue – Actual colour

It is measured as an angle from 0° to 360° on a color wheel. Red is at 0°, green is at 120°, and blue is at 240°.



Saturation – Excitation Purity. The amount of white light mixed with hue.

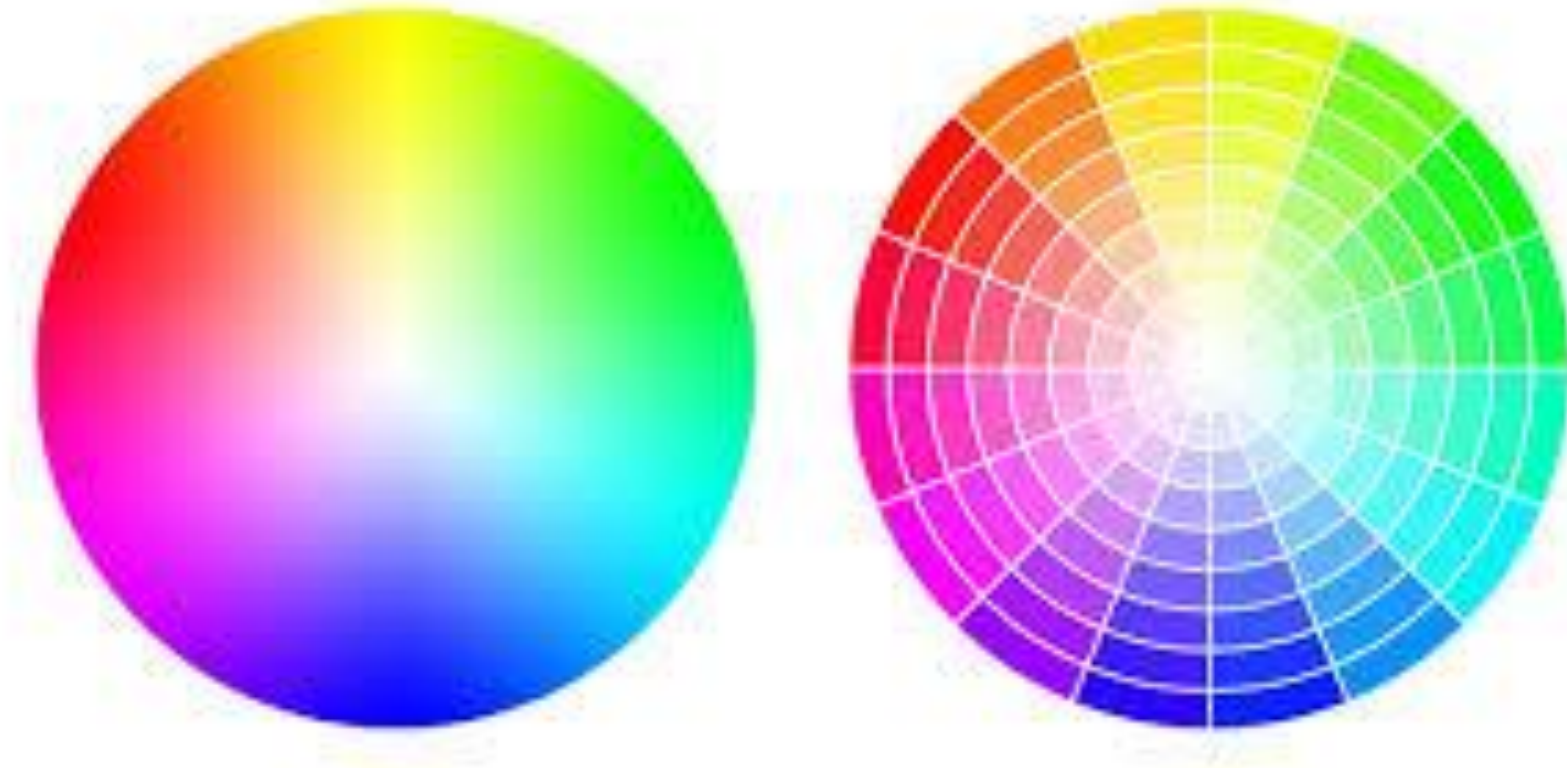
The range is from 0 (completely washed-out gray) to 1 (pure, vivid color)



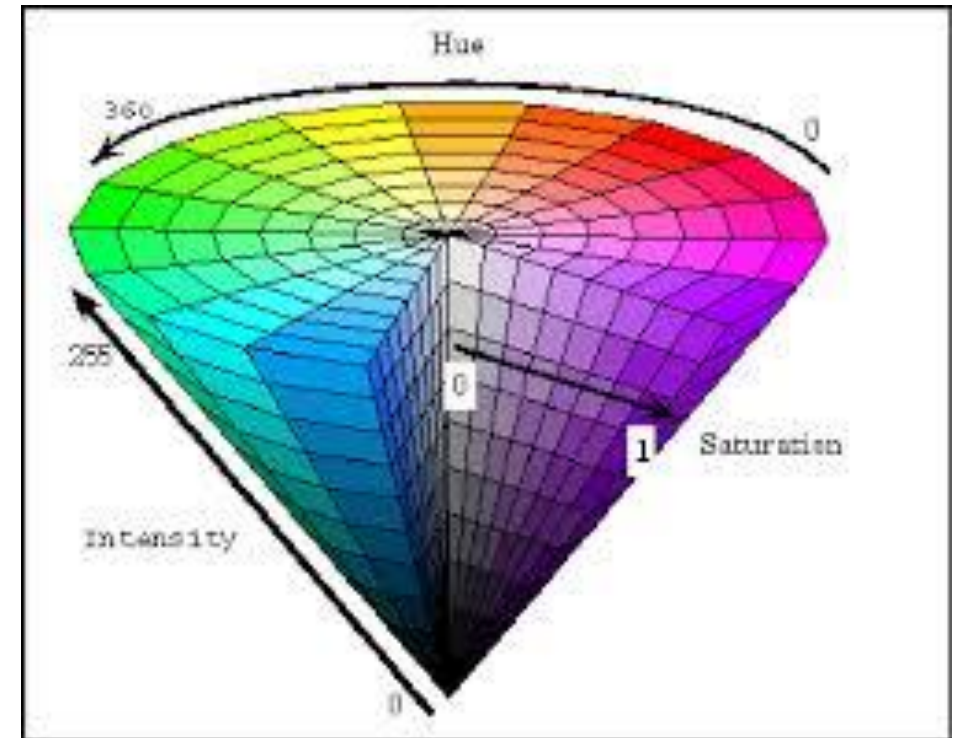
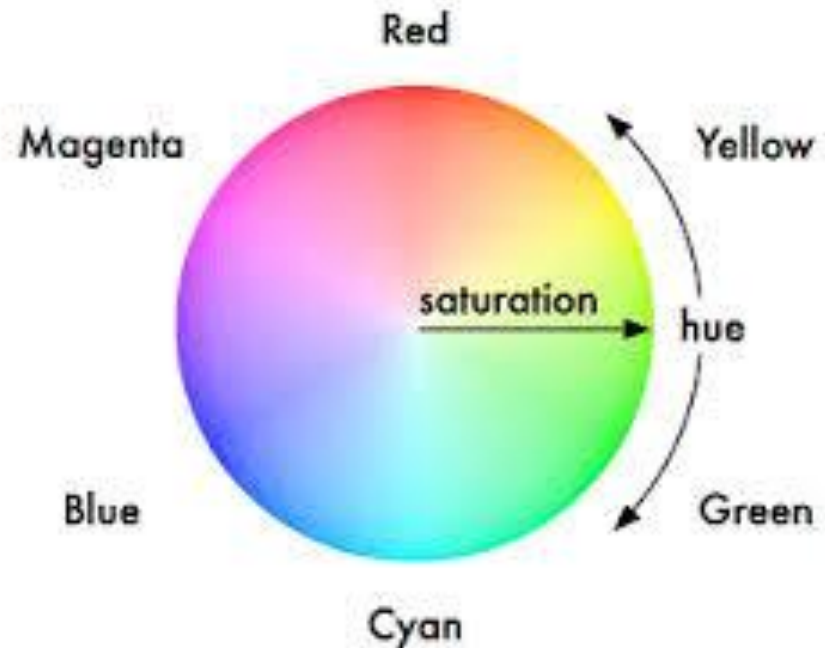
Intensity – Intensity of the light.

The range is from 0 (black) to 1 (white).

Color Wheel



HSI Model



Reference Link: <https://www.geeksforgeeks.org/electronics-engineering/what-is-hsi-color-space/>

Advantages of HSI

Better Image Analysis

HSI separates brightness from color. This makes it easier to analyze images, like finding edges or recognizing objects, because we can focus on brightness without worrying about color.

Improved Image Editing

In HSI, you can change brightness or contrast without messing up the colors. This is useful for making pictures look better or for removing noise (unwanted random pixels) from images.

Works Well in Different Lighting

HSI handles changes in lighting better. For example, it can recognize a face in both bright sunlight and in the shade because it separates the color information from the light information.

Precise Color Adjustments

You can adjust just the color without changing the brightness. For example, you can make colors more vivid or shift colors from blue to green without affecting how light or dark the image is.

Disadvantage of HSI

Hard to Change Between Models

- Changing colors from the regular way computers show colors (RGB) to HSI and back can be hard and take time. This makes things slower, especially when you need to process many images quickly.

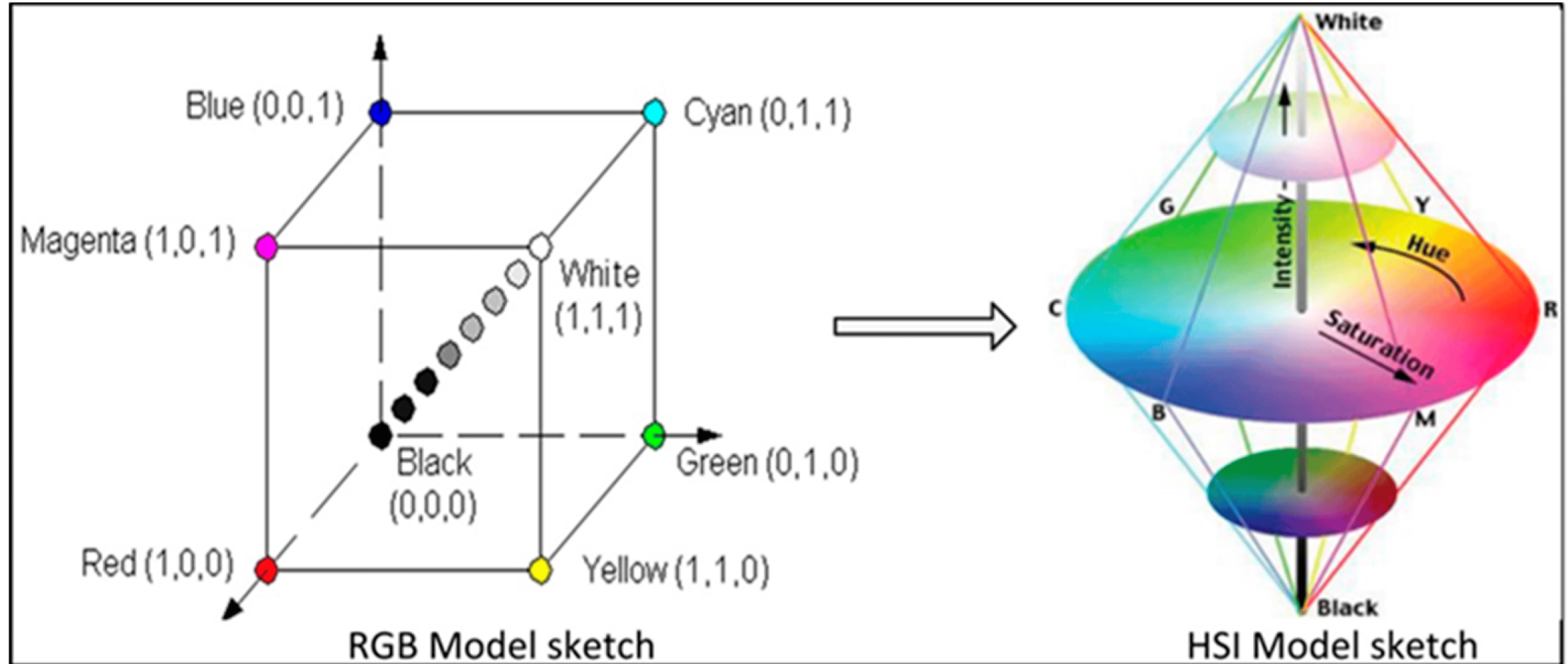
Confusing Colors

- When colors are very dull or almost gray, the HSI model can get confused and not work as well. It has trouble figuring out the exact color in these cases.

Not Always Supported

- Many software programs and tools are made to work with RGB, not HSI. This means you might have to do extra work to use HSI in some programs.
-

RGB To HSI Conversion



RGB To HSI Conversion

Steps to be followed:

1. Read a RGB image
2. Represent the RGB image in the range [0 1]
3. Find HSI components

$$\theta = \cos^{-1} \left\{ \frac{\frac{1}{2}[(R-G) + (R-B)]}{\left[(R-G)^2 + (R-B)(G-B)^{1/2} \right]} \right\}$$

$$4. H(\text{Hue}) = \begin{cases} \theta & \text{If } B \leq G \\ 360 - \theta & \text{If } B > G \end{cases}$$

$$5. S(\text{Saturation}) = 1 - \frac{3}{(R + G + B)} [\min(R, G, B)]$$

$$6. I(\text{Intensity}) = \frac{1}{3} (R + G + B)$$

RGB To HSI Conversion

- $H = \theta \rightarrow \text{if } B \leq G$ $\cos^{-1}$ represents values from 0° to 180° (the top half of the color wheel).
 - $H = 360 - \theta \rightarrow \text{if } B > G$
 - Color Code
 - 0 degree – Red
 - 120 degree – Green
 - 240 degree – Blue
 - 60 degree – Yellow
 - 300 degree – Magenta
-

RGB To HSI Conversion Example

- Convert RGB to HSI in range [0,1]- R =24, G=98, B=118

Sol: Normalize R,G and B

$$R = 24/255 = 0.09$$

$$G = 98/255 = 0.38$$

$$B = 118/255 = 0.46$$

$$I = \frac{1}{3}(R + G + B)$$

$$= \frac{1}{3}(0.09 + 0.38 + 0.46) = \frac{1}{3} * 0.93 = 0.31$$

$$S = 1 - \frac{3}{R+G+B} \min(R, G, B)$$

$$\begin{aligned} &= 1 - \frac{3}{0.09 + 0.38 + 0.46} \min(0.09, 0.38, 0.46) = 1 - \frac{3}{0.93} * 0.09 = 1 - \frac{0.09}{0.31} = 1 - 0.29 \\ &= 0.71 \end{aligned}$$

RGB To HSI Conversion Example

- Convert RGB to HSI in range [0,1]- R =24, G=98, B=118

To Calculate Hue, First we need to find out the θ .

$$\theta = \cos^{-1}\left(\frac{\frac{1}{2}((R - G) + (R - B))}{\sqrt{(R - G)^2 + (R - B)(G - B)}}\right)$$

$$\theta = \cos^{-1}\left(\frac{\frac{1}{2}((0.09 - 0.38) + (0.09 - 0.46))}{\sqrt{(0.09 - 0.38)^2 + (0.09 - 0.46)(0.38 - 0.46)}}\right)$$

$$\theta = \cos^{-1}\left(\frac{\frac{1}{2}((-0.29) + (-0.37))}{\sqrt{(-0.29)^2 + (-0.37)(-0.08)}}\right) = \cos^{-1}\frac{-0.33}{\sqrt{0.1137}} = \cos^{-1}\frac{-0.33}{0.33}$$

RGB To HSI Conversion Example

- Convert RGB to HSI in range [0,1]- R =24, G=98, B=118

To Calculate Hue, First we need to find out the θ .

$$\theta = \cos^{-1}\left(\frac{-0.33}{0.33}\right) = \cos^{-1}(-1) = 180^\circ$$

As $B > G$, so $H = 360 - \theta$

$$\begin{aligned} H &= 360 - 180 \\ &= 180^\circ \end{aligned}$$

Array Vs Matrix Operations

- Array product

$$\begin{array}{|c|c|} \hline a1 & a2 \\ \hline a3 & a4 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline b1 & b2 \\ \hline b3 & b4 \\ \hline \end{array} = \begin{array}{|c|c|} \hline a1b1 & a2b2 \\ \hline a3b3 & a4b4 \\ \hline \end{array}$$

- Matrix product

$$\begin{array}{|c|c|} \hline a1 & a2 \\ \hline a3 & a4 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline b1 & b2 \\ \hline b3 & b4 \\ \hline \end{array} = \begin{array}{|c|c|} \hline a1b1 + a2b3 & a1b2 + a2b4 \\ \hline a3b1 + a4b3 & a3b2 + a4b4 \\ \hline \end{array}$$

Array Vs Matrix Operations

- Square the image segment

a1	a2
a3	a4

 $=$

a1 x a1	a2 x a2
a3 x a3	a4 x a4

- Multiply any number in an image segment

$*n* *$

a1	a2
a3	a4

 $=$

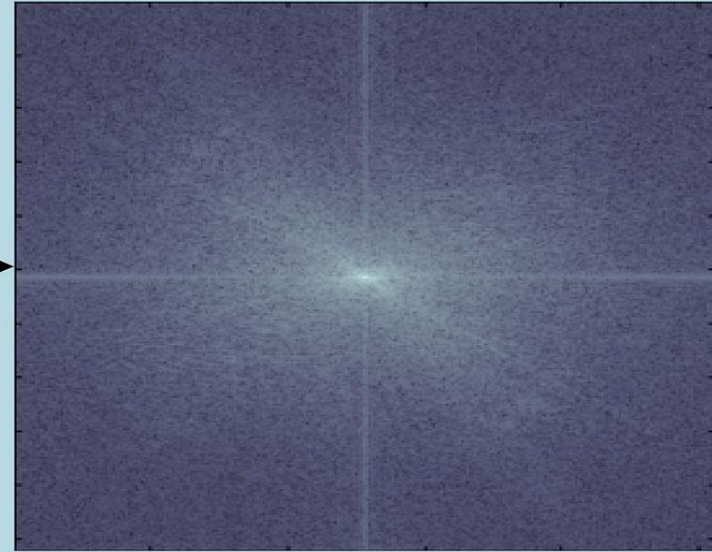
$n * a1$	$n * a2$
$n * a3$	$n * a4$

Spatial and Frequency Domain



Spatial Domain

Fourier Transform



Frequency Domain

Spatial Domain

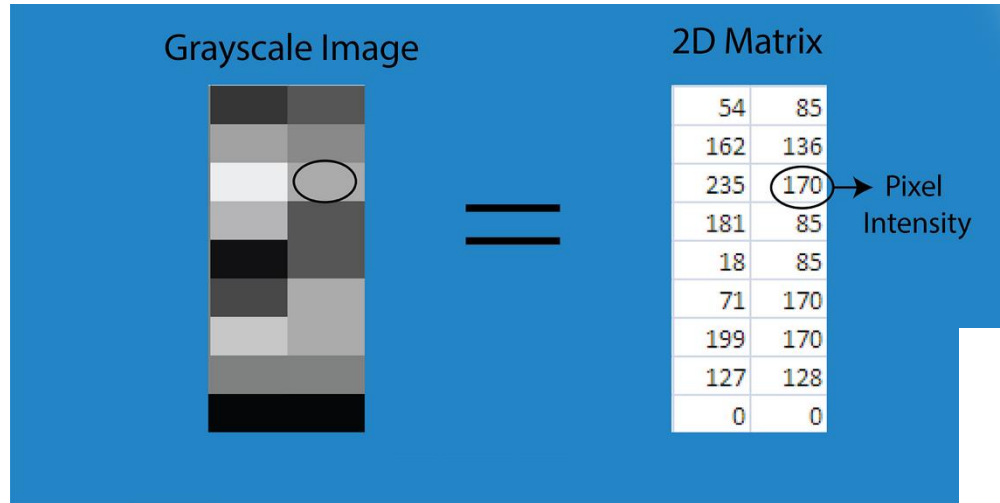
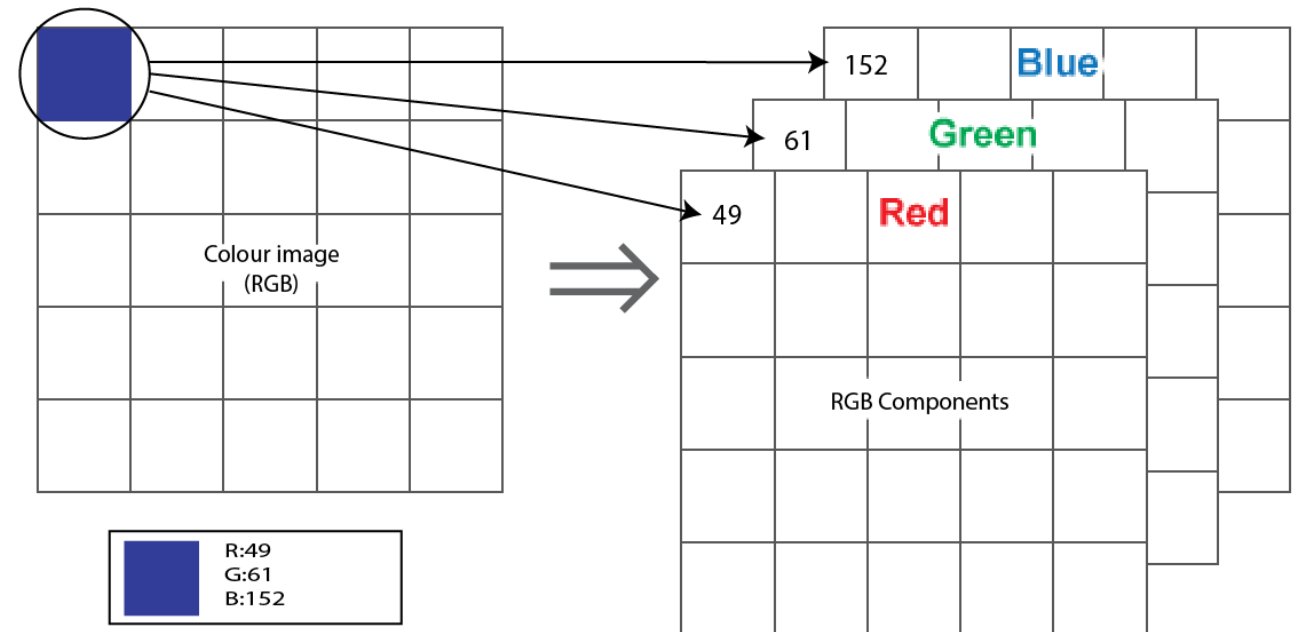


Illustration of Spatial Domain

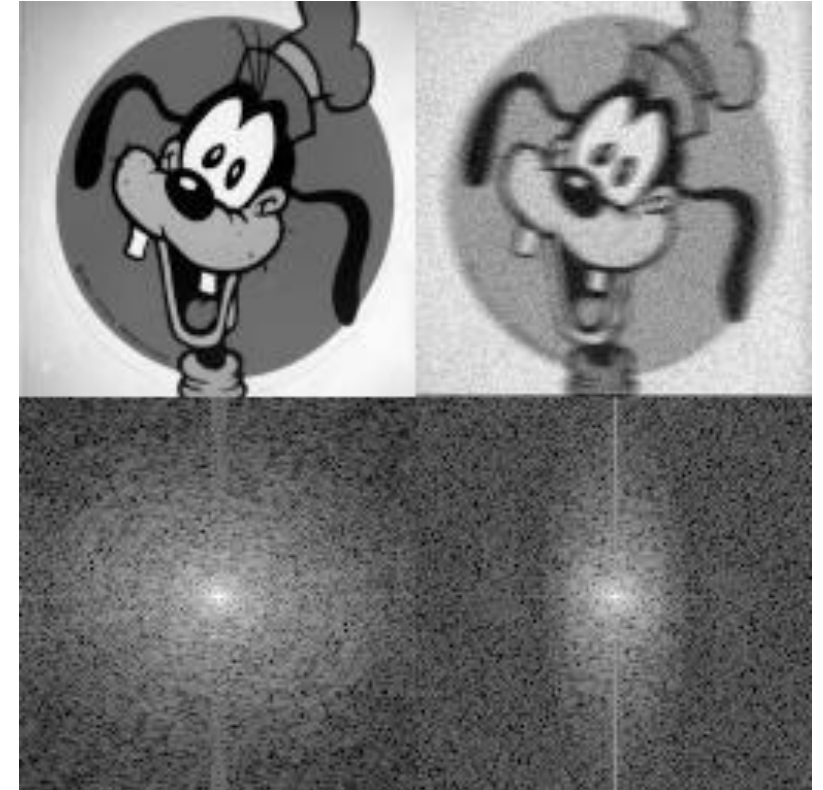
An image can be represented in the form of a 2D matrix where each element of the matrix represents pixel intensity.



Spatial domain for color image(RGB)

Frequency Domain

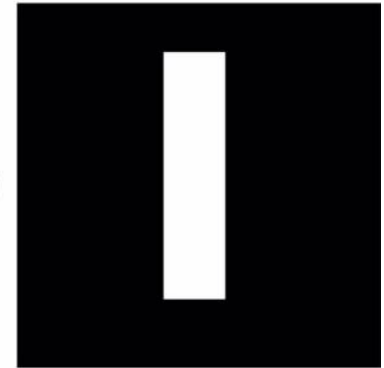
- In frequency-domain methods are based on Fourier Transform of an image.
- Roughly, the term frequency in an image tells about the rate of change of pixel values.
- Frequency domain filtering excels at addressing global image properties, like removing periodic noise, enhancing edges at specific scales, or filtering out specific frequency bands.
- Large kernels in the spatial domain can be computationally expensive, while in the frequency domain, convolution becomes multiplication, which is much faster.
- The frequency domain provides insights into the periodic patterns or textures in an image, which cannot be easily observed in the spatial domain.



Difference between Spatial Domain and Frequency Domain

Spatial domain :

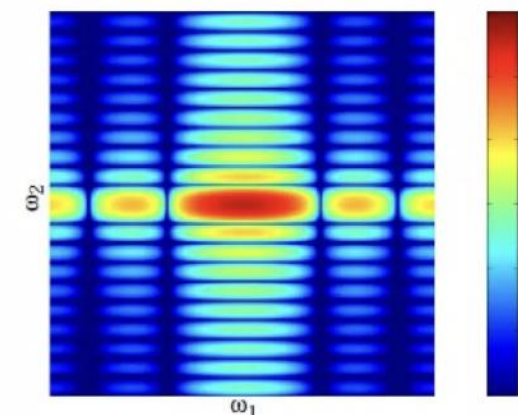
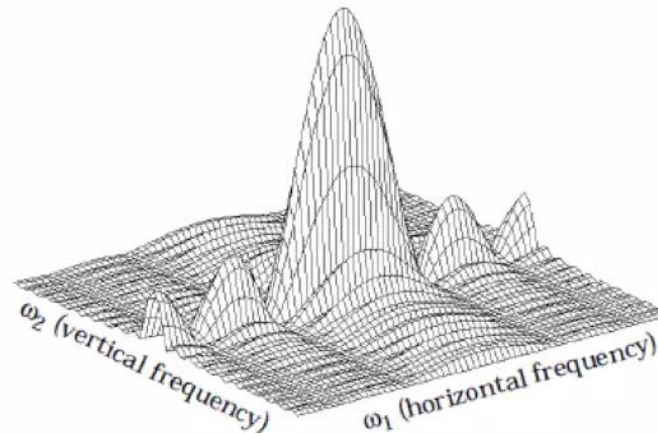
- Deal with images as it is.
- The value of the pixels of the image change with respect to scene.



Frequency domain :

- Deal with the rate at which the pixel values are changing in spatial

domain

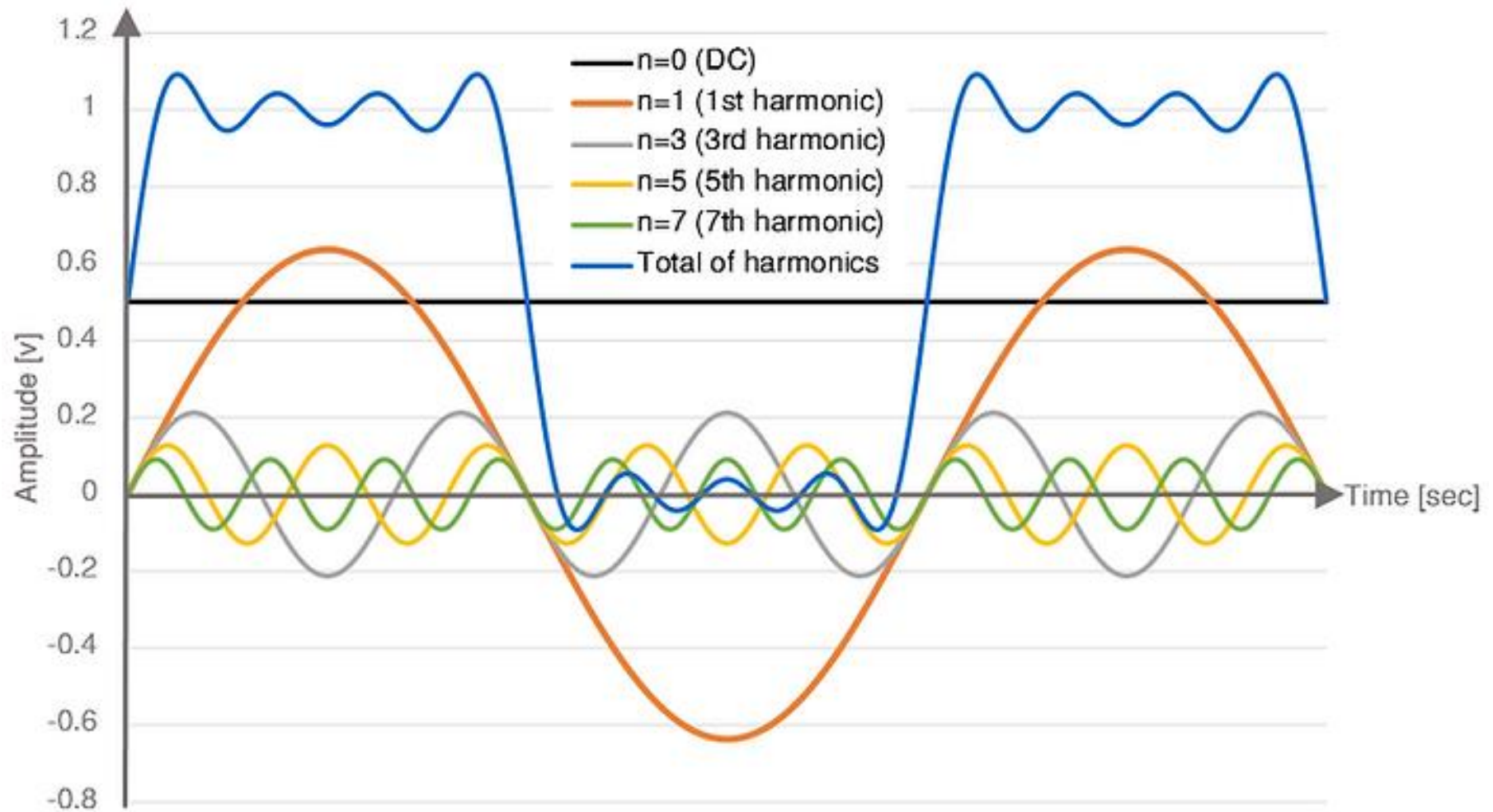


2 D Transforms

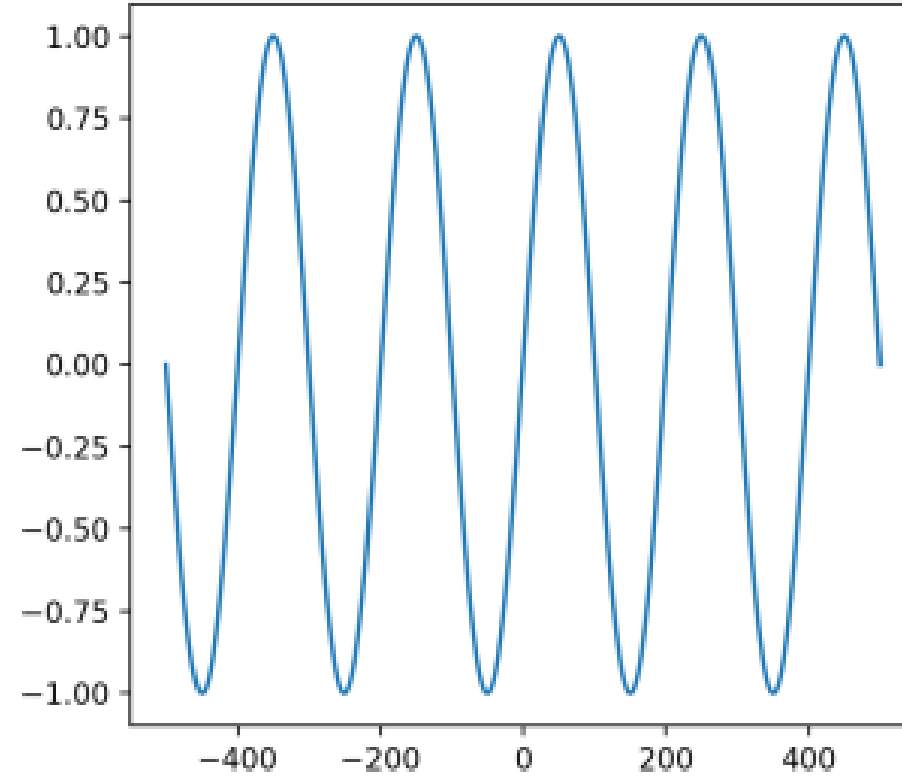
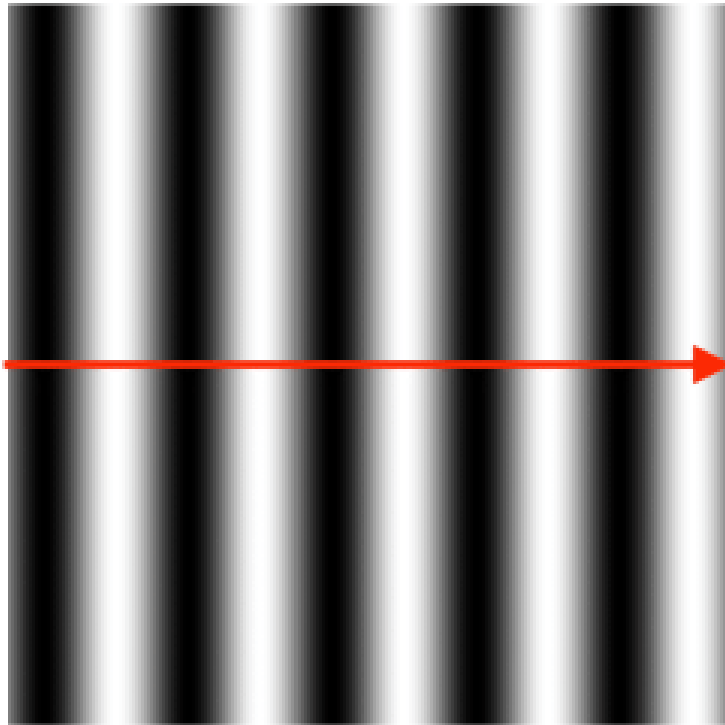


Frequency Domain

- Frequencies are the sum of sinusoid

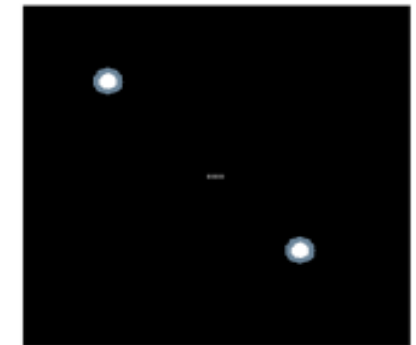
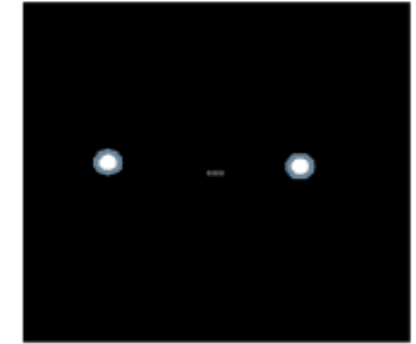
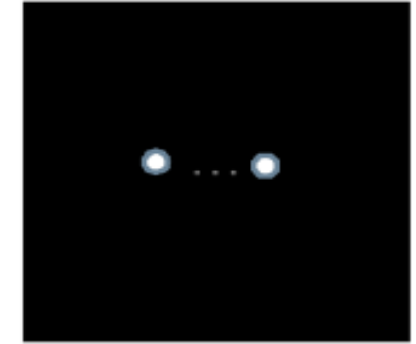
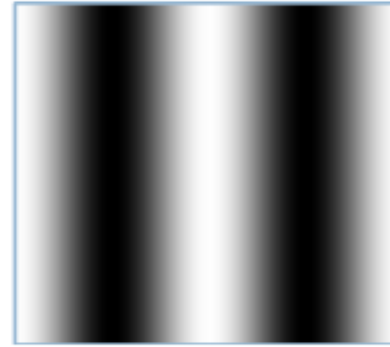


2 D Transforms

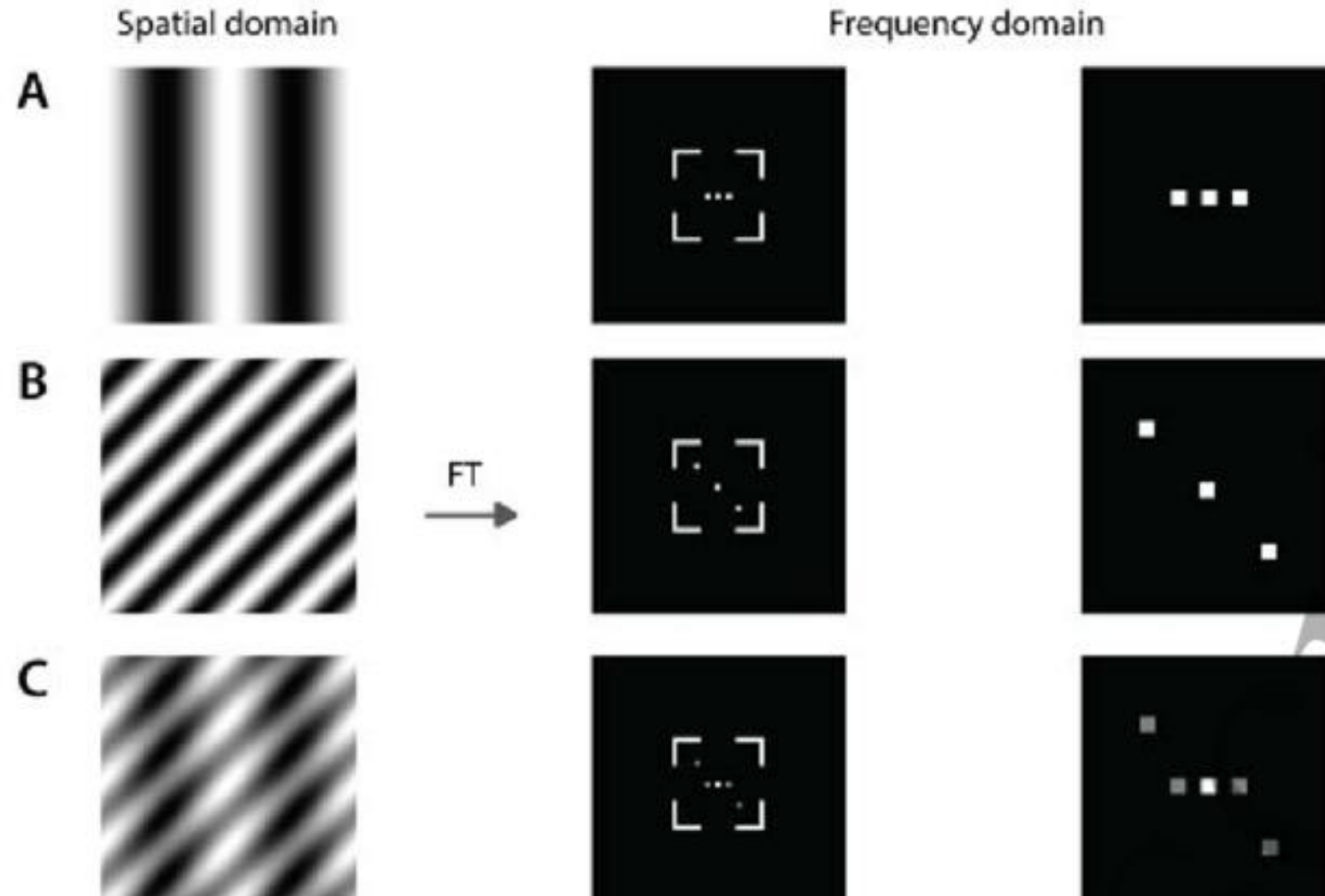


2 D Transforms

- Bright dots in the spectrum representing the dominant frequency and orientation of the patterns.
- The direction of the dots in the spectrum is always perpendicular to the direction of the stripes in the image

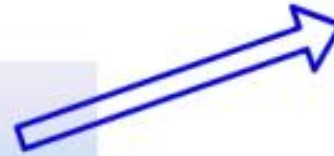
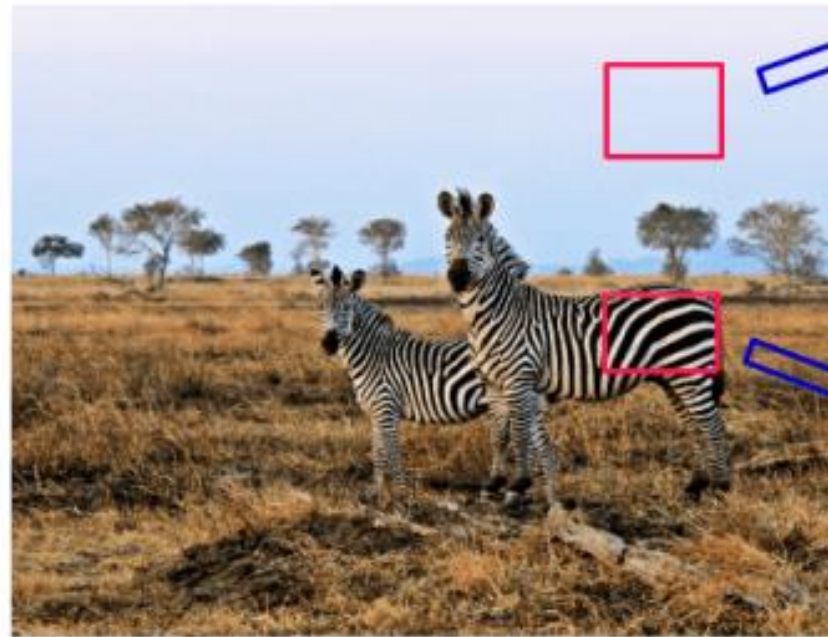


2 D Transforms



High Frequency vs Low Frequency

Edges in an image represents high frequency because the intensity changes drastically across an edge.

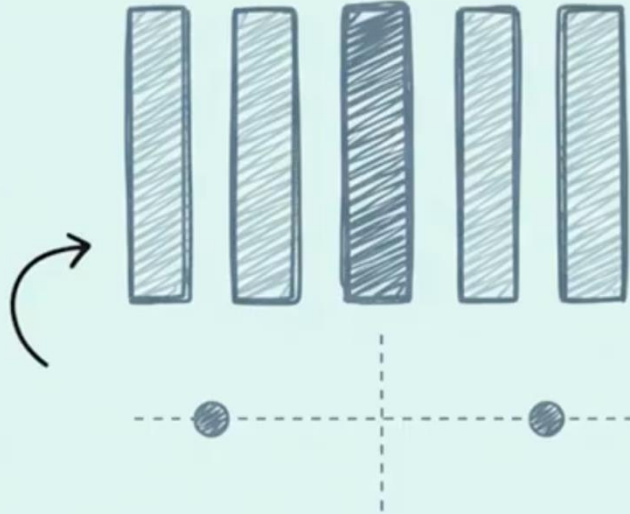


Low Frequency

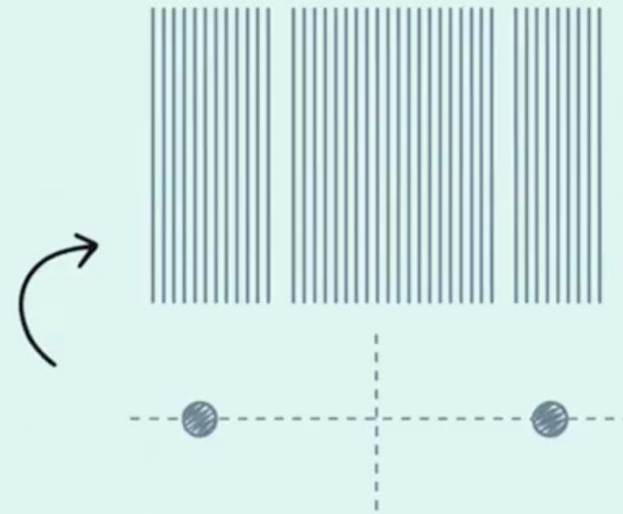


High Frequency

High Frequency vs Low Frequency



Low Frequency (wide stripes) → Dots close to center



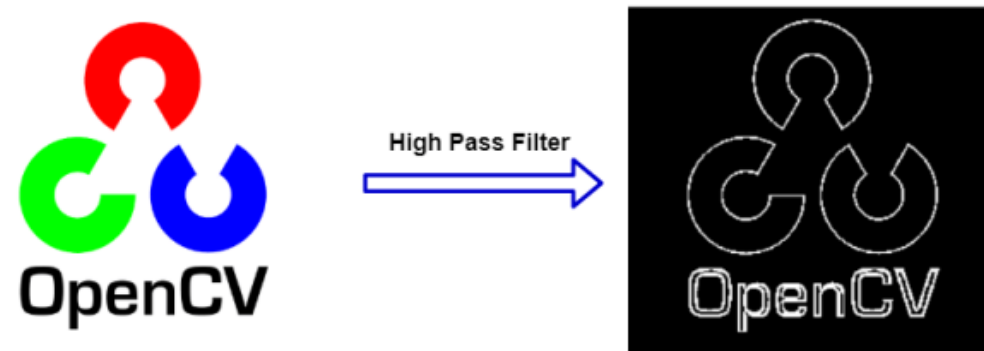
High Frequency (narrow stripes) → Dots far from center

Low Pass Filters vs High Pass Filters

Low Pass filters block high-frequency parts of an image and thus results in blurring or image smoothing. This is shown below



High Pass filter enhances high-frequency parts of an image (i.e. edges) and thus results in image sharpening



Transformation

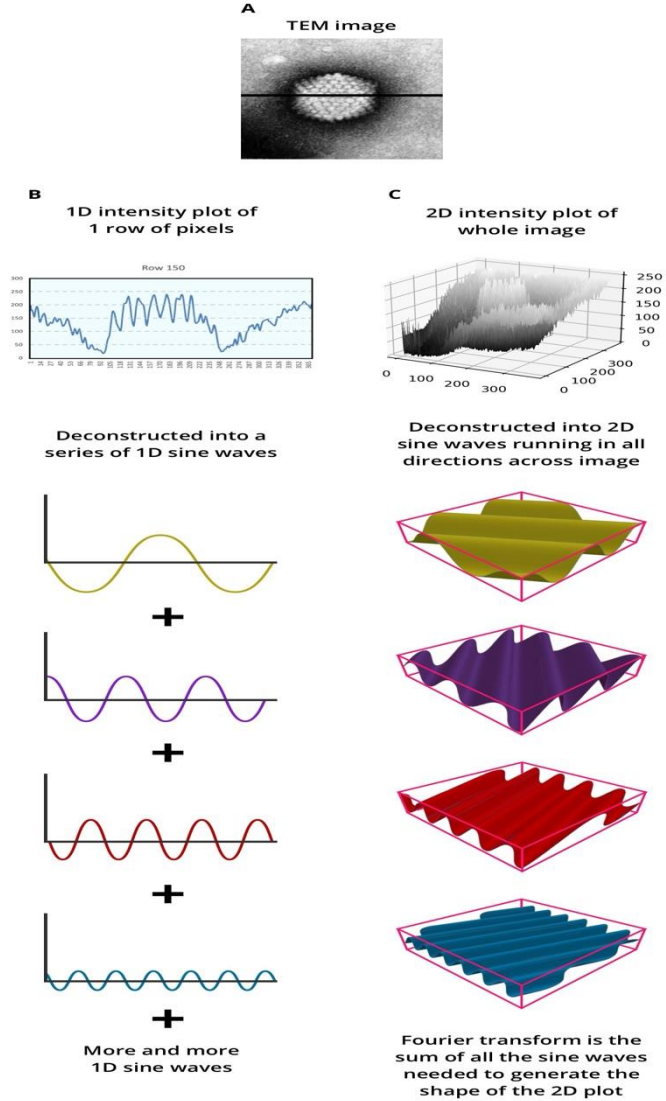
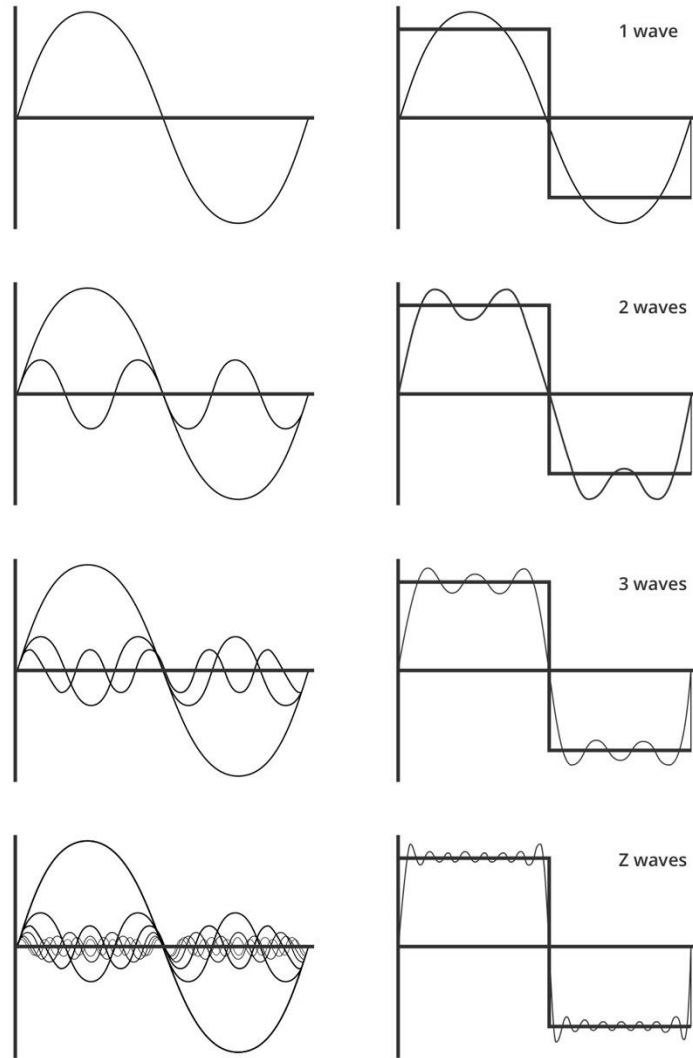
Transformation:

A signal can be converted from spatial domain into frequency domain using mathematical operators called transformation.

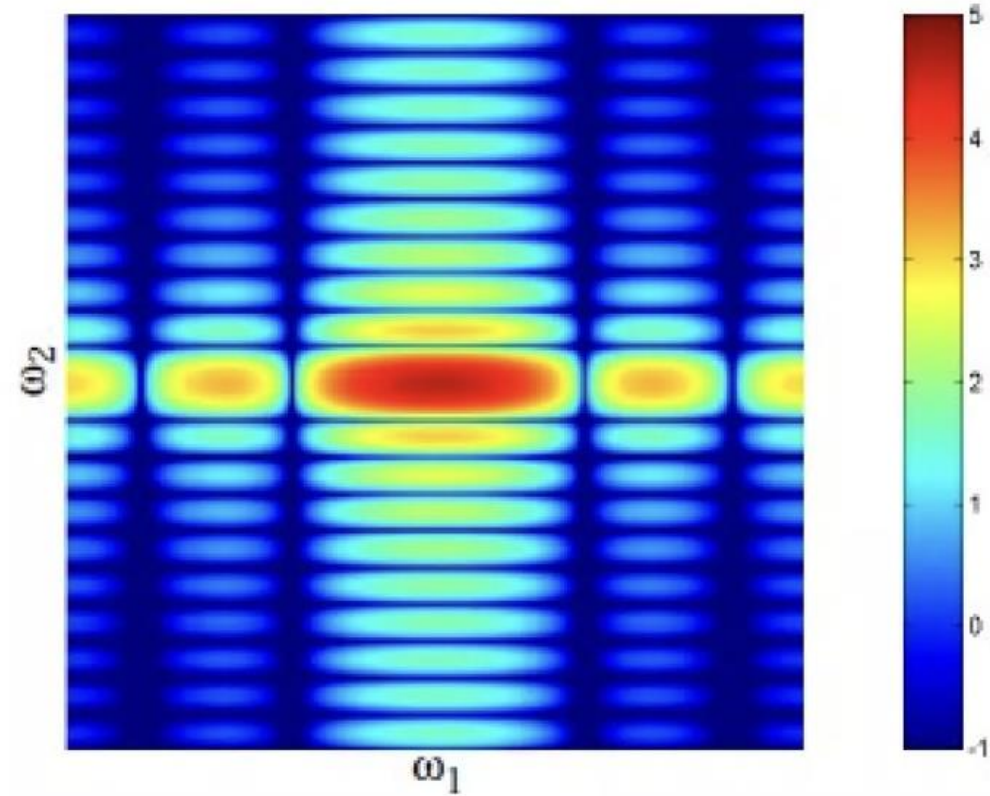
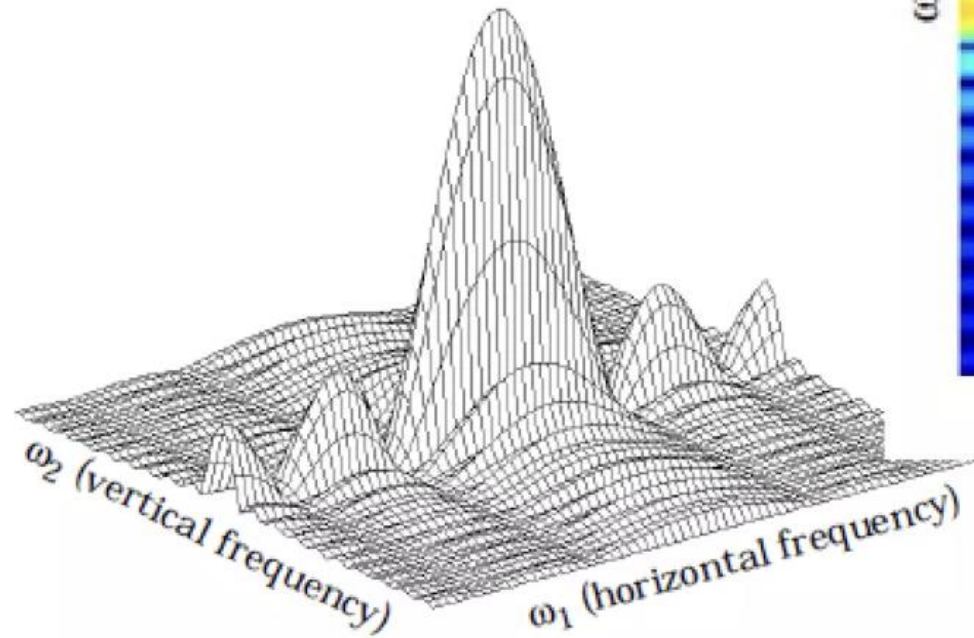
kind of transformation:

- ✚ Fourier Series
 - ✚ ***Fourier transformation***
 - ✚ Laplace transform
 - ✚ Z transform
-

Fourier Transform



Fourier Transform



Fourier Transform

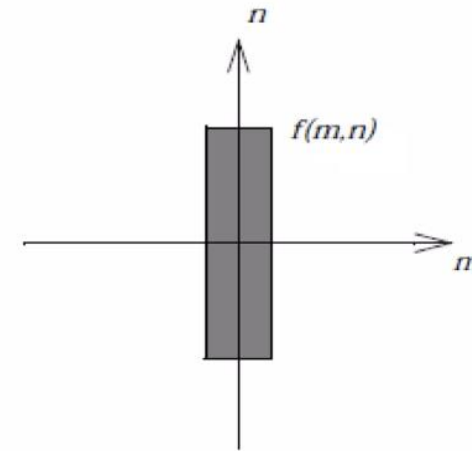
$f(m, n)$ is a function of two discrete spatial variables m and n ,

Two-dimensional Fourier transform of $f(m, n)$:

$$F(\omega_1, \omega_2) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} f(m, n) e^{-j\omega_1 m} e^{-j\omega_2 n}$$

$F(\omega_1, \omega_2)$ is often called the **frequency-domain representation of $f(m, n)$**

The variables ω_1 and ω_2 are frequency variables
 ω_1 and ω_2 both are periodic with period 2π
Where, $-2\pi \leq \omega_1$ and $-2\pi \leq \omega_2$



Fourier Transform

The inverse two-dimensional Fourier transform is given by:

$$f(m, n) = \frac{1}{2\pi} \int_{\omega_1 = -\pi}^{\pi} \int_{\omega_2 = -\pi}^{\pi} F(\omega_1, \omega_2) e^{j\omega_1 m} e^{j\omega_2 n} d\omega_1 d\omega_2$$

This equation means that $f(m, n)$ can be represented as:

A sum of an infinite number of complex exponentials (sinusoids) with different frequencies.

2-D Discrete Fourier Transformation

- The discrete two-dimensional Fourier transform of an image array is defined in series form as

$$F[k, l] = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] e^{-j2\pi \left(\frac{k}{M}m + \frac{l}{N}n \right)}$$

- inverse transform

$$f[m, n] = \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] e^{j2\pi \left(\frac{k}{M}m + \frac{l}{N}n \right)}$$

- Because the transform kernels are separable and symmetric, the two dimensional transforms can be computed as sequential row and column one-dimensional transforms.
 - The basis functions of the transform are complex exponentials that may be decomposed into sine and cosine components.
-

2-D Discrete Fourier Transformation

- It is also possible to define DFT as follows

$$F[k, l] = \frac{1}{\sqrt{MN}} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] e^{-j2\pi \left(\frac{k}{M}m + \frac{l}{N}n \right)}$$

where $k = 0, 1, \dots, M-1$ and $l = 0, 1, \dots, N-1$

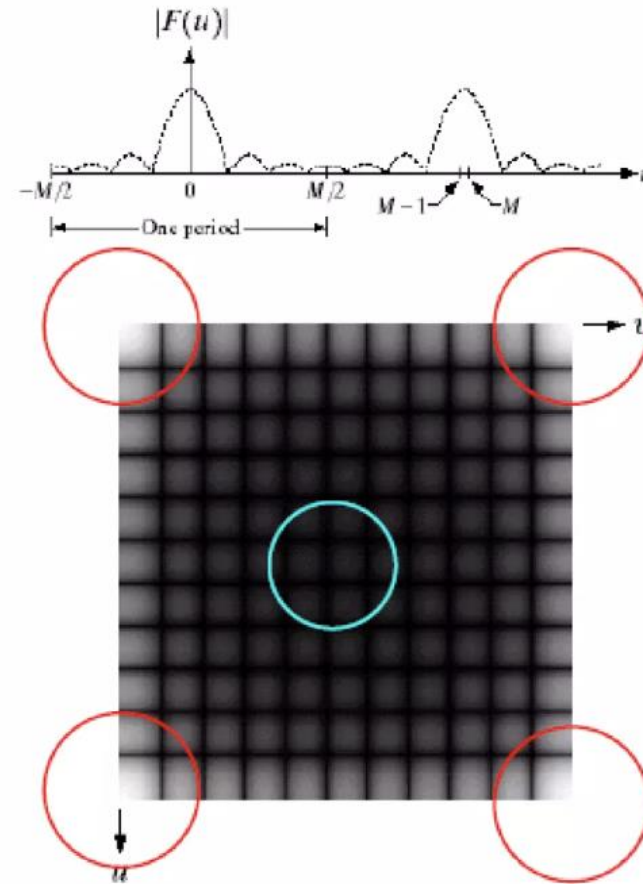
- Inverse DFT

$$f[m, n] = \frac{1}{\sqrt{MN}} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] e^{j2\pi \left(\frac{k}{M}m + \frac{l}{N}n \right)}$$

Conventional Display for 2D DFT

$F(u,v)$ has low frequency areas at corners of the image while high frequency areas are at the center of the image which is inconvenient to interpret.

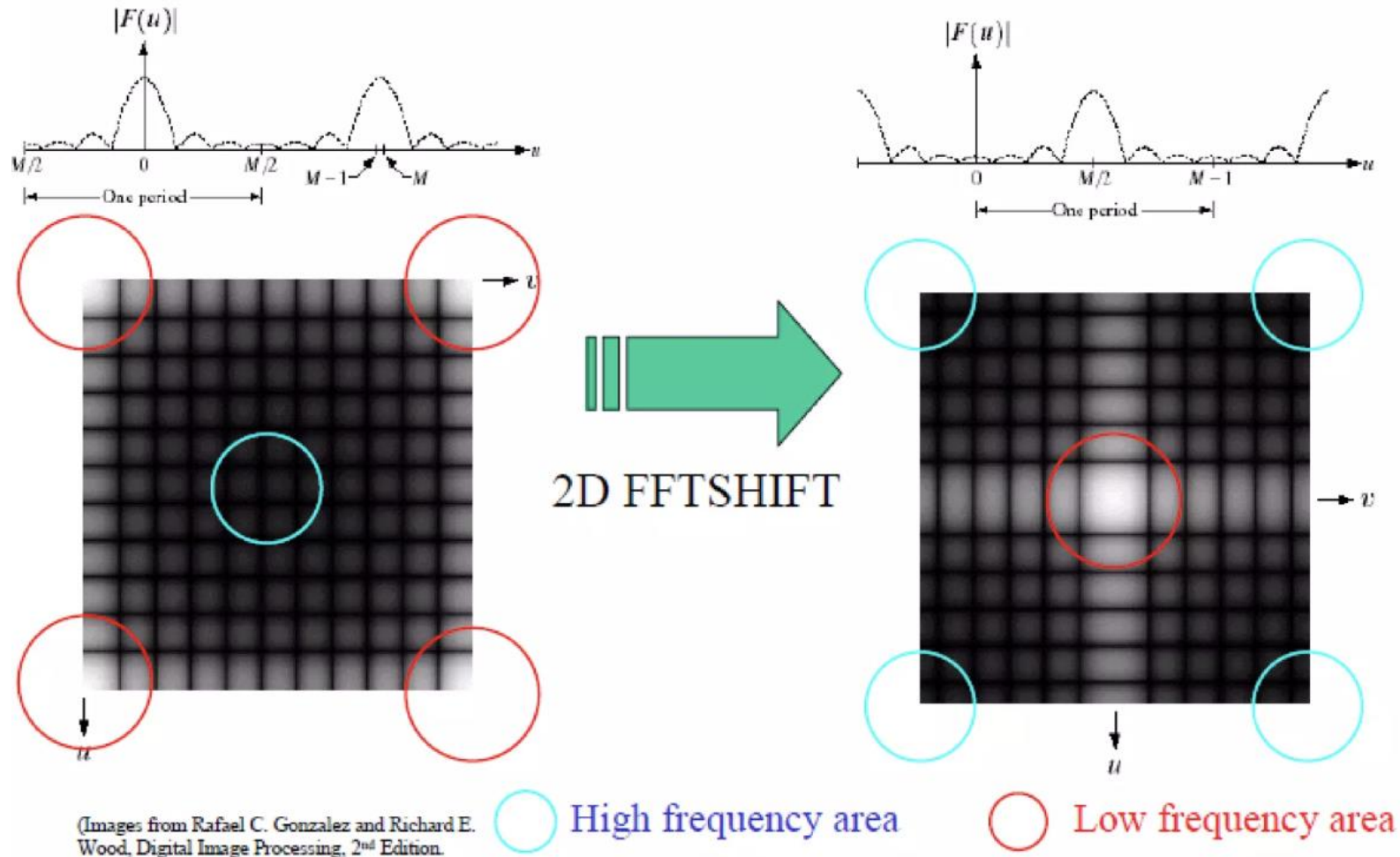
- High frequency area
- Low frequency area



(Images from Rafael C. Gonzalez and Richard E. Wood, Digital Image Processing, 2nd Edition.

2D FFT Shift: Better Display for 2D DFT

2-D FFT Shift is a MATLAB function: Shift the zero frequency of $F(u,v)$ to the center of an image.



Important Formulas

- $e^{-j\theta} = \cos\theta - j\sin\theta$
- $e^{j\theta} = \cos\theta + j\sin\theta$
- $e^{j\pi} = -1$
- $e^{-j\pi} = -1$
- $e^{j\pi/2} = j$
- $e^{-j\pi/2} = -j$
- $e^{-j2\pi} = 1$
- $e^{-j3\pi} = -1$
- $e^{-j3\pi/2} = j$
- $e^{-j9\pi/2} = -j$

Kernel of a 4 point DFT

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

1D DFT : $F[u] = \text{kernel} * f(x)$

2D DFT : $F[u,v] = \text{kernel} * f(x) * \text{kernel}^T$

Compute 2D DFT of 4x4 gray scale image

• $f(x,y) =$

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

Sol: $F(u,v) = \text{kernel} \times f(x,y) \times [\text{kernel}]^T$

DFT basis function(kernel) for N=4 is

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

Compute 2D DFT of 4x4 gray scale image

- Sol: $F(u,v) = \text{kernel} \times f(x,y) \times [\text{kernel}]^T$

- $F(u,v) =$

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

 $\times$

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

- $F(u,v) =$

4	4	4	4
0	0	0	0
0	0	0	0
0	0	0	0

 $\times$

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $=$

16	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Compute 2D IDFT of 4x4 gray scale image

• $F(u,v) =$

16	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Sol: $f(x,y) = \frac{1}{MN} [\text{kernel}]^T \times F(u,v) \times \text{kernel}$

DFT basis function(kernel) for $M=4$ and $N = 4$ is

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

Compute 2D DFT of 4x4 gray scale image

- Sol: $f(x,y) = \frac{1}{MN} [\text{kernel}]^T \times F(u,v) \times \text{kernel}$

- $f(x,y) = \frac{1}{4 \times 4} \times$

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

16	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

 $\times$

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

- $F(u,v) = \frac{1}{16} \times$

16	0	0	0
16	0	0	0
16	0	0	0
16	0	0	0

 $\times$

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

Compute 2D DFT of 4x4 gray scale image

$$\bullet F(u,v) = \frac{1}{16} \times \begin{array}{|c|c|c|c|} \hline 16 & 16 & 16 & 16 \\ \hline 16 & 16 & 16 & 16 \\ \hline 16 & 16 & 16 & 16 \\ \hline 16 & 16 & 16 & 16 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 \\ \hline \end{array}$$

Compute 2D DFT of 4x4 gray scale image using separability property

• $f(x,y) =$

0	1	2	1
1	2	3	2
2	3	4	3
1	2	3	4

Sol: $F(u,v) = \text{kernel} \times f(x,y) \times [\text{kernel}]^T$

DFT basis function(kernel) for N=4 is

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

Compute 2D DFT of 4x4 gray scale image using separability property

Sol: Calculate 1-D DFT along rows

- DFT of 1st row =

0	1	2	1
1	2	3	2
2	3	4	3
1	2	3	4

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

0
1
2
1

 $=$

4
-2
0
-2

Compute 2D DFT of 4x4 gray scale image using separability property

Sol: Calculate 1-D DFT along rows

- DFT of 2nd row =

0	1	2	1
1	2	3	2
2	3	4	3
1	2	3	4

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

1
2
3
2

 $=$

8
-2
0
-2

Compute 2D DFT of 4x4 gray scale image using separability property

Sol: Calculate 1-D DFT along rows

- DFT of 3rd row =

0	1	2	1
1	2	3	2
2	3	4	3
1	2	3	4

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

2
3
4
3

 $=$

12
-2
0
-2

Compute 2D DFT of 4x4 gray scale image using separability property

Sol: Calculate 1-D DFT along rows

- DFT of 4th row =

0	1	2	1
1	2	3	2
2	3	4	3
1	2	3	4

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

1
2
3
2

 $=$

8
-2
0
-2

Compute 2D DFT of 4x4 gray scale image using separability property

- Therefore, 1D DFT along rows is

4	-2	0	-2
8	-2	0	-2
12	-2	0	-2
8	-2	0	-2

Compute 2D DFT of 4x4 gray scale image using separability property

Now, Calculate 1-D DFT along columns

- DFT of 1st column =

4	-2	0	-2
8	-2	0	-2
12	-2	0	-2
8	-2	0	-2

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

4
8
12
8

 $=$

32
-8
0
-8

Compute 2D DFT of 4x4 gray scale image using separability property

Now, Calculate 1-D DFT along columns

- DFT of 2nd column =

4	-2	0	-2
8	-2	0	-2
12	-2	0	-2
8	-2	0	-2

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

-2
-2
-2
-2

 $=$

-8
0
0
0

Compute 2D DFT of 4x4 gray scale image using separability property

Now, Calculate 1-D DFT along columns

- DFT of 3rd column =

4	-2	0	-2
8	-2	0	-2
12	-2	0	-2
8	-2	0	-2

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

0
0
0
0

 $=$

0
0
0
0

Compute 2D DFT of 4x4 gray scale image using separability property

Now, Calculate 1-D DFT along columns

- DFT of 4th column =

4	-2	0	-2
8	-2	0	-2
12	-2	0	-2
8	-2	0	-2

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

 $\times$

-2
-2
-2
-2

 $=$

8
0
0
0

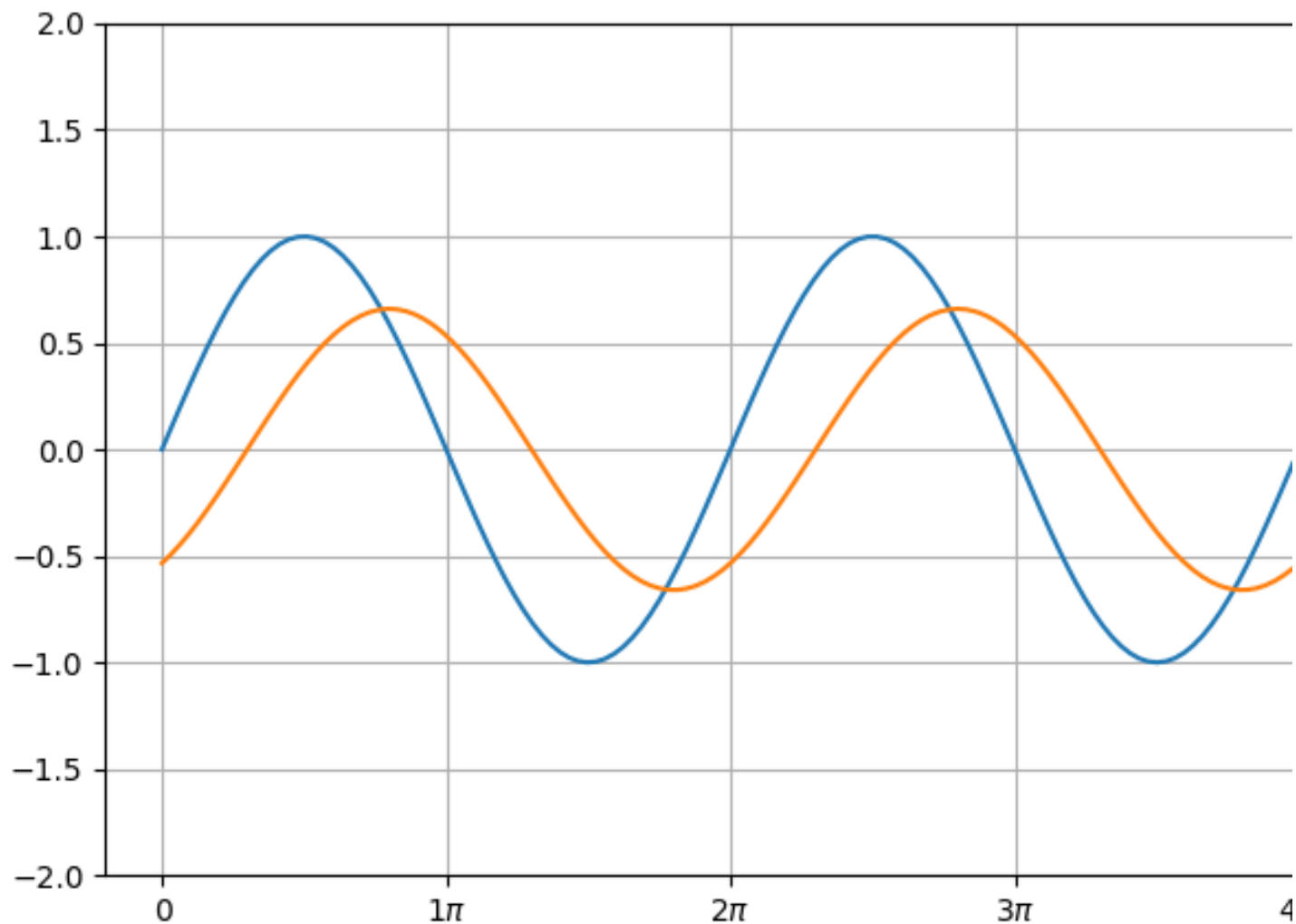
Compute 2D DFT of 4x4 gray scale image using separability property

- Hence, final solution is

32	-8	0	-8
-8	0	0	0
0	0	0	0
-8	0	0	0

2-D Discrete Cosine Transformation

- The discrete cosine transform (DCT) is a mathematical technique that breaks down a data sequence or an image into a weighted sum of cosine waves oscillating at different frequencies.



2-D Discrete Cosine Transformation

- Unlike the discrete Fourier transform (DFT) which uses complex numbers with both sine and cosine components, the DCT uses exclusively real-valued cosine functions.
- It mirrors data at the boundaries to avoid abrupt jumps or discontinuities, making it more efficient for finite signals.
- It packs most of the important information of a signal or image into just a few low-frequency coefficients.
- **Applications**
 - Image Compression
 - Video Coding
 - Audio Compression



2-D Discrete Cosine Transformation

Comparision to DFT:

- As compared to DFT, application of DCT results in less blocking artifacts due to the even symmetric extension properties of DCT.
- DCT uses real computations, unlike the complex computations used in DFT.
- DFT is a complex transform and therefore stipulates that both image magnitude and phase information be encoded.
- DCT hardware simpler, as compared to that of DFT.
- DCT provides better energy compaction than DFT for most natural images.
- The implicit periodicity of DFT gives rise to boundary discontinuities that result in significant high-frequency content. After quantization, *Gibbs Phenomenon* causes the boundary points to take on erroneous values .

2-D Discrete Cosine Transformation

- The forward equation, for image A, is

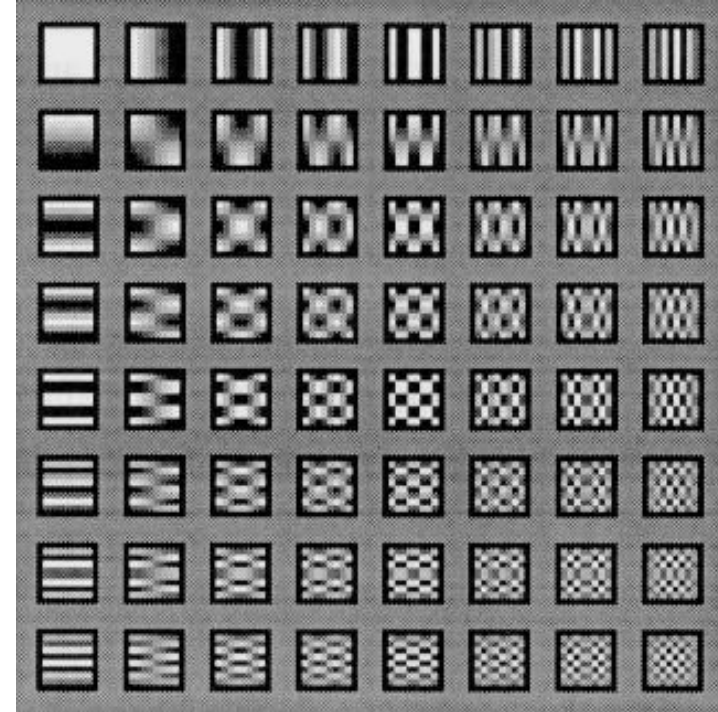
$$b(u, v) = \frac{2}{N} C(u) C(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} a(x, y) \cos\left(\frac{\pi u(2x+1)}{2N}\right) \cos\left(\frac{\pi v(2y+1)}{2N}\right)$$

- The inverse equation, for image B, is

$$a(x, y) = \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} C(u) C(v) b(u, v) \cos\left(\frac{\pi u(2x+1)}{2N}\right) \cos\left(\frac{\pi v(2y+1)}{2N}\right)$$

• Here $C(u) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } u = 0 \\ 1 & \text{otherwise} \end{cases}$

DCT Basis function



2-D Discrete Cosine Transformation

For $N=4$ basis function C is calculated as

0.5	0.5	0.5	0.5
0.653	0.2705	-0.2705	-0.653
0.5	-0.5	-0.5	0.5
0.2705	-0.653	0.653	-0.2705

Find DCT of given 4x4 image

• $f(x,y) =$

2	4	4	2
4	6	8	3
2	8	10	4
3	8	6	2

Sol: $F(u,v) = C \times f(x,y) \times C^T$

DCT basis function(Cosine) for N=4 is

0.5	0.5	0.5	0.5
0.653	0.2705	-0.2705	-0.653
0.5	-0.5	-0.5	0.5
0.2705	-0.653	0.653	-0.2705

Compute 2D DCT of 4x4 gray scale image

- Sol: $F(u,v) = C \times f(x,y) \times C^T$
- $F(u,v) =$

0.5	0.5	0.5	0.5
0.653	0.2705	-0.2705	-0.653
0.5	-0.5	-0.5	0.5
0.2705	-0.653	0.653	-0.2705

×

2	4	4	2
4	6	8	3
2	8	10	4
3	8	6	2

×

0.5	0.653	0.5	0.2705
0.5	0.2705	-0.5	-0.653
0.5	-0.2705	-0.5	0.653
0.5	-0.653	0.5	-0.2705

Compute 2D DCT of 4x4 gray scale image

- Sol: $F(u,v) = C \times f(x,y) \times C^T$

- $F(u,v) =$

1.9	-0.2705	-8	0.653
-2.6913	-0.2498	2.3087	0.8957
-3.5	1.4645	1.5	-1.6885
0.0327	-1.6022	-0.9562	-0.2448